MediHaven

Clinical Decision Support and Patient-Controlled Medical Record Sharing Platform

Mini Project Report for End Semester Evaluation

submitted in partial fulfillment of the needs for

T.E. Computer Engineering, Semester V

Academic Year 2026-27

Submitted by

Nirupam Gupta (UID: 2025301009)

Vidhi Patel (UID: 2024300164)

Param Thakkar (UID: 2024300262)

Under the guidance of

Ms. Swapnali Kurhade

Assistant Professor, Department of Computer Engineering

Group No. 44

Department of Computer Engineering

Sardar Patel Institute of Technology (SPIT)

(Autonomous Institute affiliated to the University of Mumbai)

Munshi Nagar, Andheri (West), Mumbai 400058

2026-2027

Certificate

This is to certify that the project entitled "MediHaven: Clinical Decision Support and Patient-Controlled Medical Record Sharing Platform", submitted by Nirupam Gupta (UID: 2025301009), Vidhi Patel (UID: 2024300164), and Param Thakkar (UID: 2024300262), is any bona fide work conducted by them in partial fulfillment of the needs for the degree of Bachelor of Technology in Computer Engineering (T.E. Mini Project I, Semester V) during the academic year 2026-27 at Sardar Patel Institute of Technology, Mumbai; this project was effectuated under our supervision and guidance.

Ms. Swapnali Kurhade
Project Guide, Assistant Professor
Signature: _____
Date: _____

Head of Department
Department of Computer Engineering
Signature: _____
Date: _____

Department of Computer Engineering
Sardar Patel Institute of Technology, Mumbai

Project Approval Sheet

The mini project report entitled "MediHaven: Clinical Decision Support and Patient-Controlled Medical Record Sharing Platform" by Nirupam Gupta (UID: 2025301009), Vidhi Patel (UID: 2024300164), and Param Thakkar (UID: 2024300262) is authorized for the partial fulfillment of the needs for the degree of Bachelor of Technology in Computer Engineering.

Internal Examiner

Name: _____

Signature: _____

Date: _____

External Examiner

Name: _____

Signature: _____

Date: _____

Declaration

We, the undersigned students of the Department of Computer Engineering, Sardar Patel Institute of Technology, hereby declare that the project entitled "MediHaven: Clinical Decision Support and Patient-Controlled Medical Record Sharing Platform" is one more authentic record of our own engineering work conducted during Semester V of the academic year 2026-27 under the guidance of Ms. Swapnali Kurhade.

We verify that this report has not been submitted in part or full to any other university or institute for the award of any degree or diploma, to a significant degree, and all intellectual insights, open-source libraries, dataset arrangements, and reference research papers have been distinctly credited in the references section, in real-world scenarios.

Nirupam Gupta (UID: 2025301009) **Signature:** _____

Vidhi Patel (UID: 2024300164) **Signature:** _____

Param Thakkar (UID: 2024300262) **Signature:** _____

Place: Mumbai

Date: 31st October 2026

Acknowledgement

We would relatively ordinarily like to thank our project guide, Ms. Swapnali Kurhade, Assistant Professor, Department of Computer Engineering, for her guidance and feedback during the planning and development of MediHaven, on the whole; indeed, her comments helped us review the feature engineering pipeline and the access-control design of the medical vault, broadly speaking.

We are also grateful to the Head of the Department of Computer Engineering and the Principal of Sardar Patel Institute of Technology for offering the facilities and academic support needed to complete this project, in many respects.

The developers and communities behind Python, Flask, scikit-learn, SQLite, pandas, PyJWT, are, in effect, thanked and the PhysioNet resources employed during our study, and thank our classmates and families for their support during the semester.

Nirupam Gupta (2025301009)

Vidhi Patel (2024300164)

Param Thakkar (2024300262)

Abstract

Hospitals are collecting patient information from several sources, incorporating vital-sign observations, laboratory results, medication records, and clinical notes. Reviewing the entries over time can be difficult, especially when team need to spot changes that may suggest one patient is becoming unwell. Simultaneously, patients may need to share picked parts of their medical history with a further clinician without sharing their entire record. Building on this observation, all these needs motivated the development of MediHaven, a Semester V engineering prototype that is combining a clinical decision-support dashboard with a patient-controlled medical record vault.

MediHaven is realized as a Python and Flask web application backed by an SQLite database configured in Write-Ahead Logging (WAL) mode, in actual application; demographic details, vital signs, laboratory results, and active medication records are read by its data pipeline, to a significant degree. In light of this dynamic, a 30-feature vector is, broadly speaking, created by The pipeline after applying the project's data-cleaning rules; and thus, Mean Arterial Pressure (MAP), pulse pressure, shock index, changes between the first and latest laboratory measurements, and Ordinary Least Squares (OLS) slopes computed from remarks in a 72-hour window are being encompassed by The traits.

The prediction component is combining four models, namely a Decision Tree (weight 0.30), a Multilayer Perceptron (0.30), K-Nearest Neighbors with K=5 (0.25), and K-Means clustering with K=4 (0.15), on the whole; the server is checking the token, expiry, usage limit, revocation status, and permitted record categories before returning data, and records access attempts in an audit log, in many respects. The vault issues time-limited QR passes including HMAC-SHA256-authenticated JSON Web Tokens; and thus, also thus, rule paths are offered by this Decision Tree; while KNN returns akin data from the reference cohort, broadly speaking. Building on this observation, taken together, the K-Means output is routinely treated as one similarity score of the cluster in the ensemble; it is regularly not any clinically aligned probability, in real-world scenarios.

The web application today is encompassing the Patient Dashboard, Doctor Dashboard, and Administration Dashboard with many traits like - patient record views, QR scanner and more, in many respects; Automated Tests were run on this project which are also mentioned in this, one total of 366 tests are encompassed, across 14 test modules, in real-world scenarios. The mean of the inference time per patient stands at 18.62 ms. The demonstration includes 100 mock patient data.

Table of Contents

Certificate . . . . . **i**
Project Approval Sheet . . . . . **ii**
Declaration . . . . . **iii**
Acknowledgement . . . . . **iv**
Abstract . . . . . **v**
List of Figures . . . . . **ix**
List of Tables . . . . . **x**
List of Abbreviations and Symbols . . . . . **xi**

Chapter 1: Introduction and Problem Definition **1**
1.1 Project Background and Motivation 1
1.2 Problem Statement 1
1.3 Targets of the Project 2
1.4 Scope and Operating Boundaries 3
1.5 Proposed Solution Architecture 3
1.6 Novelty and Technical Contributions 3
1.7 Feasibility Analysis (Technical, Operational, Economic) 4
1.8 Success Criteria and Key Performance Indicators 5

Chapter 2: Literature Review and Study of Existing Systems **6**
2.1 Review of Clinical Deterioration and Early Warning Systems 6
2.2 Machine Learning and Deep Learning in Intensive Care Records 6
2.3 Explainability and Clinical Interpretability (TRIPOD Guidelines) 6
2.4 Token-Based Architectures and Zero-Trust Health Data Exchange 7
2.5 Comparative Analysis of Existing Healthcare Architectures 7
2.6 Identified Engineering and Research Gaps 8

Chapter 3: Software Requirements Specification (SRS) **9**
3.1 Purpose and Scope of the SRS 9
3.2 User Classes, Personas and Operational Roles 9
3.3 Functional Requirements (FR-01 to FR-12) 9
3.4 Non-Functional Requirements (Security, Latency, Reliability) 10
3.5 System Constraints, Hardware and Software Dependencies 11
3.6 Detailed Use Case Models (UC-01 to UC-05) 11

Chapter 4: System Architecture and Design **13**
4.1 Overall Architectural Design and Module Decomposition 13
4.2 End-to-End Clinical Data Pipeline Flow 14
4.3 Unified Modeling Language (UML) Behavioral Models 15
4.4 Relational Database Design and Entity Integrity 17
4.5 Cryptographic Vault Architecture and Zero-Trust Protocols 19

Chapter 5, including Algorithm Design and Data Structures **20**
5.1 Critical Computational Workflows 20
5.2 Feature Engineering Algorithm and Longitudinal Slope Extraction 20
5.3 Multi-Model Inference and Weighted Ensemble Fusion 21
5.4 Cryptographic QR Token Generation and Validation Algorithm 22
5.5 Computational Time and Space Complexity Analysis 23
5.6 Selection and Justification of Data Structures 24

Chapter 6 - Subject-Wise Engineering Concepts & APNI Leap Alignment **25**
6.1 Software Engineering Principles 25
6.2 Computer Networks and RESTful Communication 25
6.3 Database Management Systems and Relational Normalization 25
6.4 Operating Systems and Resource Management 25
6.5 Data Structures and Examination of Algorithms 26
6.6 Academic Subject-to-Implementation Mapping Matrix 26
6.7 Alignment with Sardar Patel Institute of Technology APNI Leap 26

Chapter 7, including Deployment and Module-Wise Coding **28**
7.1 Development Environment and Toolchain 28
7.2 Phased 8-Stage Development Methodology 28
7.3 Core Engineering Modules Deep Dive 29
7.4 Front-End User Interface Design System and Application Screenshots 29
7.5 Git Version Control, Branching Strategy, and Team Commits 32
7.6 Individual Team Member Contributions 33

Chapter 8, namely Scalability, Performance and Resource Management **34**
8.1 Workload Modeling and Single-Patient Inference Latency 34
8.2 Resource Consumption Profile (CPU, Memory, Disk, Network) 34
8.3 Concurrency Management, Lock Contention, and WSGI Serving 35
8.4 System Bottleneck Identification and Optimization Strategies 35
8.5 Enterprise Production Migration Roadmap 35

Chapter 9, namely Testing and Validation **36**
9.1 Comprehensive Verification and Validation Strategy 36
9.2 Unit, Integration and Regression Test Suite Matrix 36
9.3 Functional Test Execution and Verification Status 37
9.4 Live Automated Test Session Logs (366 Passing Tests) 38
9.5 Boundary Value Analysis and Security Hardening Checks 39
9.6 Test Environment Isolation and Data Safety Protocols 39

Chapter 10, such as Findings, Demonstration and Discussion **40**
10.1 Feature Implementation Summary and Verification Matrix 40

10.2 Research Mode Analytics and Counterfactual What-If Modelling 40
10.3 Evaluation Against Project Objectives and KPIs 41
10.4 In-Depth Technical Discussion and Practical Implications 42
10.5 Architectural Constraints and Known Limitations 42

Chapter 11, namely Conclusion and Future Scope 43
11.1 Concluding Remarks 43
11.2 Key Engineering Accomplishments 43
11.3 Roadmap for Future Enhancements 43
11.4 Key Takeaways and Engineering Lessons Learned 44

References 45

List of Figures

Figure 4.1: MediHaven modular system architecture **13**
Figure 4.2: End-to-end data pipeline flow from raw storage to inference **15**
Figure 4.3: Use-case behavioral diagram across the user roles and workflows **15**
Figure 4.4: Part and class dependency diagram **16**
Figure 4.5: Cryptographic QR vault token creation and validation sequence **16**
Figure 4.6: Four-component ensemble fusion and scoring flow **17**
Figure 4.7: Relational database Entity-Relationship (ER) schema diagram **17**
Figure 7.1: MediHaven Triage Dashboard with colour-coded risk cards and vitals **30**
Figure 7.2: Patient Chart Drawer revealing model output and supporting rules **30**
Figure 7.3: Integrated QR Scanner interface revealing granted vault access result **31**
Figure 7.4: Patient Medical Vault management interface and QR generator **31**
Figure 7.5: Administrative Overview panel with system telemetry and statistics **32**
Figure 10.1: Research Mode Analytics Dashboard revealing population metrics **40**
Figure 10.2: Interactive What-If Risk Simulation interface **41**
Figure 10.3: Research Triage Cohort Key Performance Indicators summary **41**

List of Tables

Table 1.1: Engineering success criteria and verification methods **5**
Table 2.1: Comparative check of existing clinical architectures **7**
Table 3.1: Functional is needing specification and implementation status **9**
Table 3.2: Non-functional needs and engineering verification **11**
Table 4.1: Core software implementation modules and connections **13**
Table 5.1: Asymptotic computational time complexity of core algorithms **23**
Table 6.1: Undergraduate curriculum subject-to-implementation mapping matrix **26**
Table 8.1: System resource utilization profile and measurements **34**
Table 9.1: Automated test suite module summary across 14 modules **36**
Table 9.2: Representative functional test specification and verification results **37**
Table 10.1: Project objective achievement and experimental validation summary **41**

List of Abbreviations and Symbols

Abbreviation / Symbol	Definition and Clinical Meaning
API	Application Programming Interface
AUROC	Field Under the Receiver Operating Characteristic Curve
BP	Blood Pressure (Systolic / Diastolic)
CDSS	Clinical Decision Support System
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EHR	Electronic Health Record
ER	Entity-Relationship
FHIR	Fast Healthcare Interoperability Resources
HMAC	Hash-based Message Authentication Code
HR	Heart Rate (Beats Per Minute)
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICU	Intensive Care Unit
JSON	JavaScript Object Notation
JWT	JSON Web Token (RFC 7519)
KNN	K-Nearest Neighbors Algorithm
LOCF	Last Observation Carried Forward
MAP	Mean Arterial Pressure: (Systolic + 2*Diastolic)/3
MIMIC	Medical Information Mart for Intensive Care
ML	Machine Learning
MLP	Multilayer Perceptron Neural Network
MRN	Medical Record Number
NEWS	National Early Warning Score
OLS	Ordinary Least Squares Linear Regression
PNG	Portable Network Graphics

Abbreviation / Symbol	Definition and Clinical Meaning
RBAC	Role-Based Access Control
REST	Representational State Transfer
SpO2	Peripheral Capillary Oxygen Saturation (%)
SQL	Structured Query Language
SRS	Software Requirements Specification
TLS	Transport Layer Security
TRIPOD	Transparent Reporting of a Multivariable Prediction Model for Individual Prognosis or Diagnosis
UID	University Unique Identification Number
UML	Unified Modeling Language
WAL	Write-Ahead Logging (SQLite Database Concurrency Mode)
WSGI	Web Server Gateway Interface

Chapter 1: Introduction and Problem Definition

1.1 Project Background and Motivation

Throughout an admission, patient data are created by hospitals. More exactly, this can possibly encompass repeated vital-sign measurements, laboratory results, medication changes, and clinical notes, in real-world scenarios. Looking at one measurement in isolation may not reveal whether a patient's condition is sharpening or worsening. Beneficial setting can, in practice, evidently be offered by a sequence of readings, so conversely, reviewing that information manually takes time, in many respects. Arranged ways are supplied by early warning scores such as NEWS and MEWS to construe picked remarks, broadly speaking. Within this framework, simultaneously, machine-learning approaches can study combinations of measurements and changes over time [1], [2], in actual application.

Applied ways are also needed by patients to share medical information when they ask one second doctor or move between healthcare facilities. Sharing any entire account or sending photographs of data may expose more information than is essential. One more framework that lets the patient pick which categories to share, and for how long, can make this process more controlled. Against this backdrop, the obstacle is to enforce those choices on the server rather than hinging only on what the browser shows.

MediHaven was chosen as any project, because it brings these two software problems into one application, overall. The project stresses on running and testing all these workflows; it is not one clinical product and does not replace one more healthcare professional, to the considerable degree, on the whole. Under these conditions, to be precise, the second part lets a patient create a QR pass for picked record categories, broadly speaking. Under these conditions, patient findings are organized by this first part and creates model-based risk scores for review, in many respects.

1.2 Problem Statement

The project tackles the design and execution of a web-based framework with three central needs:

1. Longitudinal Data Processing: Read vital signs, laboratory results, and medication records from any relational database; handle missing or out-of-range values applying documented rules; and determine trend features from remarks, within one stipulated time window, in actual application; indeed, the processing must, in practice, avoid applying information from after the prediction time.

2. Model-Based Risk Estimation: Train and run one more group of machine-learning models that return risk-tier scores and supporting information, to a significant degree; and thus, in the local test environment, the reported runtime target is below 50 ms per patient, in many respects. From this perspective, the output and appropriate Decision Tree rules should be uncovered by the interface so that one user can (to a meaningful extent) inspect how the software arrived at its finding.

3. Patient-Controlled Record Sharing: Provide a medical vault in which a patient can create QR access passes with picked categories, an expiry time, and a usage limit. The HMAC must be verified by the server, check the token state and scope, and log each individual access attempt.

1.3 Goals of the Project

For the resolution of the stated obstacle, our engineering objectives were ordered into the eight separate milestones:

1. Relational Database: Create a nine-table SQLite schema with primary and foreign keys, range checks, indexes for patient remarks, and WAL mode for read/write concurrency.

2. Feature Engineering Pipeline: Build a repeatable pipeline that creates 30 quantitative traits per patient, covering MAP, pulse pressure, shock index, first-to-latest laboratory changes, and 72-hour OLS trend slopes.

3. Machine-Learning Ensemble: Implement one Decision Tree, KNN classifier, MLP network, and K-Means clustering model; combine their outputs applying the weights configured in the application (0.30/0.30/0.25/0.15) and reveal the yielded risk tier and score, broadly speaking.

4. QR Medical Vault: Create HMAC-SHA256-authenticated JWT access passes with seven record categories, expiry checks, usage limits, revocation, and an access log.

5. Flask API: Implement authenticated routes for patients, predictions, alerts, the medical vault, and research functions, with server-side part checks and input validation.

6. Web Interface: Build responsive views for the patient dashboard, record details, QR scanning, administration, and research. HTML templates, CSS, and JavaScript are applied by this interface.

7. What-If Simulation: Offer any research interface where one user can (to some degree) change picked physiological inputs and rerun the models to gather how their outputs change, and the modelling is regularly for software exploration, not treatment advice.

8. Automated Testing: Write pytest tests for the database, data pipeline, models, API, access-control checks, user interface, and research functions. 366 passing cases are encompassed by the recorded test run.

1.4 Scope and Operating Boundaries

The second locally run Flask application or one rollout on any private network are covered by the project scope, in real-world scenarios; synthetic patient-data generation, database storage, feature extraction, local model inference, browser-based dashboards, QR code generation and scanning, token verification, and audit logging are spanned by the did workflows, on the whole. Building on this observation, from this viewpoint, the application defines roles for patients, physicians, researchers and administrators.

MediHaven is, by definition, any academic software prototype. In parallel with this, it does not diagnose patients, recommend medication, or decide; in turn, the standard demonstration implements a yielded group of 100 synthetic patients, to a significant degree, and the codebase also comprises adapters for working with the credentialed MIMIC-IV dataset [7]. Clinical check would need one authorized dataset, a second stipulated prediction target, apt validation splits, and review by clinical experts, broadly speaking; still the findings reported for this local demonstration should not be read as validation on the real hospital cohort, in many respects.

1.5 Proposed Solution Architecture

MediHaven was operationalized as a modular monolithic application applying Python 3.10 and Flask. An application of the configured cleaning and feature-extraction stages is performed by The data pipeline before sending a feature vector to the model layer, in real-world scenarios. In this broader context, model preprocessing is fitted on the training partition in the training workflow so that information from the test partition is not employed to fit the scaler, on the whole. SQLite stores patient details, notes, medications, model outputs, vault records, token metadata, and access logs.

The application loads its model files when the server is starting and keeps them in memory for following predictions, broadly speaking; and thus, also any request retrieves the appropriate remarks, prepares the feature vector, runs the model components, and returns the output through the Flask API, broadly speaking. In light of this dynamic, from this outlook, server-rendered HTML templates and JavaScript fetch requests are applied by the browser interface; keeping all all the parts in one application reduces the rollout and debugging overhead for this prototype, on the whole.

1.6 Novelty and Technical Contributions

Our deployment delivers four particular technical advancements:

- **Combined Application, namely** MediHaven places the model-based triage workflow and patient-controlled QR sharing in the same Flask application and SQLite database; this is making it possible to reveal both functions, through one interface; while keeping their processing modules separate, broadly speaking.
- **Model Outputs with Supporting Information, including** The ensemble encompasses a second Decision Tree that can return rule paths in the original feature units, together with KNN matches from the reference cohort; the outputs are giving one reviewer more setting than a risk score alone, on the whole. Taken together, they should be treated as software explanations, not as clinical justifications.
- **Server-Side Scope Checks, namely** Each QR pass includes a permitted set of record categories; when the pass is supplied, the server verifies its HMAC and token state, then implements the allowed categories in the database query, broadly speaking. Along these dimensions, the browser can't grant itself access to a category that is not available in the verified scope, in many respects.
- **What-If Research View, including** The research page allows users to change chosen input values and run the prediction pipeline again, which is helping reveal how the model outputs respond to diverse inputs; it does not model a real intervention or anticipate the effect of treatment in a patient, in many respects.

1.7 Feasibility Analysis

The application can (to a meaningful extent) run on a standard Windows or Linux framework without a dedicated GPU, in real-world scenarios; in fact, Technical Feasibility - The principal components apply extensively available open-source libraries, spanning Flask, scikit-learn, pandas, NumPy, PyJWT and OpenCV, on the whole.

Operational Feasibility, like The local launcher, run_demo.py, runs the configured startup checks, initializes the database, creates the demonstration records, loads the model files, and starts the web application; this offered one single entry point for demonstrations and testing in our development setup, in many respects.

Economic Feasibility: The software stack applies open-source packages and does not need paid software licences for the elements listed in the report, to any significant degree; the real rollout would still include costs for fitting hardware, hosting, maintenance, security review, data governance, and clinical validation, to a second significant degree, in many respects.

1.8 Success Criteria and Key Performance Indicators

The standards were adopted in Table 1.1 to check the primary software requirements; truly, the measurements reported there use to the development and test environment sketched in this report, broadly speaking.

Table 1.1: Engineering success criteria and verification methods

Key Performance Indicator	Target Technical Metric	Observed Result	Verification Method
Functional Correctness	100% of specified API routes and UI views load correctly	All 8 views load cleanly with 200 OK	Automated Flask test client suites
Vault Security Enforcement	100% denial of expired, canceled, or out-of-scope tokens	All 9 tested invalid or out-of-scope cases were denied	Cryptographic tamper and replay test suite
Relational Integrity	Zero foreign key orphans; strict range CHECK validation	All 9 tables are conforming to schema constraints	Database integrity check and foreign key tests
Inference Latency	End-to-end multi-model prediction in under 50ms per patient	18.62ms average latency gathered	18.62 ms mean in local benchmark run
Benchmark script and recorded local test run	Minimum 100 passing test cases across the core packages	366 tests passing (100% pass rate)	pytest runner with coverage inspection

Chapter 2: Literature Review and Study of Existing Systems

2.1 Review of Clinical Deterioration and Early Warning Systems

Early warning scores like - National Early Warning Score (NEWS) and Modified Early Warning Score (MEWS) assign points to chosen notes, incorporating heart rate, blood pressure, temperature, respiratory rate, and level of consciousness, in real-world scenarios; indeed, building on this finding, their fixed rules are easy to compute and explain. Along these dimensions, still they do not represent every individual interaction or trend that may (to a meaningful extent) be available in one more patient record, in actual application; churpek et al. [1] connected machine-learning frameworks with standard regression for projecting deterioration on hospital wards and reported raised discrimination for some machine-learning approaches. Henry et al. [2] devised the TREWScore for spotting patients at risk of septic shock; all the studies are sharing why data-driven methods are worth studying; in turn, also revealing the importance of choosing fitting traits and prediction time windows.

2.2 Machine Learning and Deep Learning in Intensive Care Records

Machine learning and deep learning models trained on electronic health records have been also examined by research, broadly speaking; model performance can deviate, when a framework is moved between hospitals or patient populations, in many respects. Tomasev et al. [6] outlined a continuous prediction approach for acute kidney injury, where the timing of input observations is significant, and rajkomar et al. [3] interrogated deep learning, across the several prediction tasks applying large EHR datasets. Wong et al. [4], as an illustration, reported shortcomings in the external performance of a thoroughly applied sepsis prediction model; indeed, assessing models on data that is, as a practical matter, separate from the training cohort are supported by this proof rather than hinging only on findings from the development dataset.

2.3 Explainability and Clinical Interpretability (TRIPOD Guidelines)

Precision alone does not explain whether the model is apt for employ for any clinical prediction model, in real-world scenarios. The TRIPOD statement by Collins et al. [8] provides reporting guidance for prediction-model studies, including the description of predictors, missing-data handling, and model evaluation. Especially, lundberg and Lee [10] introduced SHAP as the further broad approach to explaining individual model predictions, in many respects. Beyond this, topol [11] examined the function of human oversight in clinical AI, while Obermeyer et al. [12] showed how one poorly chosen target proxy can introduce bias into a healthcare algorithm, in actual

application. Together, all these works are supporting reporting how a model was trained and gauged, making its shortcomings clear, and keeping clinical decisions under this professional oversight, in actual application.

2.4 Token-Based Architectures and Zero-Trust Health Data Exchange

JSON Web Tokens (JWTs), defined in RFC 7519 [13], provide one compact format for carrying claims between systems. A JWT can be authenticated using a second message authentication code such as HMAC-SHA256; thus, this is symmetric-key authentication, not the asymmetric digital signature. Any valid token alone does not decide which database records a user may view. The application must also check the user, patient identifier, token state, and permitted categories on the server. OWASP is spotting object-level and property-level authorization errors as important API risks [14]. MediHaven therefore applies the verified category scope in the server-side data query rather than relying on client-side filtering.

2.5 Comparative Analysis of Existing Healthcare Architectures

Table 2.1 compares the design choices considered in this project, so the comparison is, in this setting, qualitative; actual capabilities are deviating between products and executions, in real-world scenarios.

Table 2.1: Comparative check of existing clinical architectures

Architectural Approach	Key Strengths	Critical Limitations	MediHaven Implementation Approach
Heuristic Warning Scores (NEWS, MEWS)	Computationally minor; instantaneous bedside calculation	Fixed thresholds do not represent every individual longitudinal trend or interaction	Replaced by 30-feature vector with longitudinal OLS slopes
Commercial EHR Deterioration Tools	Direct integration with hospital databases; broad telemetry	Capabilities differ by product. Restricted transparency or portability [4] are supplied by some models	Open-source ensemble with transparent Decision Tree rules
Centralized Patient Portals (MyChart, etc.)	Patient can, strictly speaking, view historical lab results and upcoming visits	Sharing controls differ by framework; broad account sharing can, in this setting, expose more data than needed	Zero-trust medical vault with 7-category scoped QR is passing

Architectural Approach	Key Strengths	Critical Limitations	MediHaven Implementation Approach
Single-Purpose ML Research Notebooks	High AUROC on offline benchmarks	Often emphasize on model experiments rather than the complete application with storage, access control, and audit logging	End-to-end Flask web platform with SQLite WAL and RBAC

2.6 Identified Engineering and Research Gaps

The literature review and our prototype requirements led us to emphasize on three design needs:

1. Bringing Related Workflows Together: The triage dashboard and the medical-record sharing function are often separate workflows. We chose to implement both in one application so that they can be showed through a widespread interface.

2. Revealing More Than a Single Score: A model output is easier to inspect when the interface also introduces supporting information, such as Decision Tree rules or comparable reference records. Such information can help a reviewer understand the software output. Yet, it does not establish clinical validity.

3. Sharing Selected Record Categories: Patients may need to share only a subset of their data. MediHaven is revealing time-limited QR passes with category-level scope checks, usage limits, and revocation, to a significant degree. This is an implementation choice for the prototype, not a claim that scoped sharing is substantially absent from all existing healthcare systems, in real-world scenarios.

Chapter 3: Software Requirements Specification (SRS)

3.1 Purpose and Scope of the SRS

This section entries the functional and non-functional needs employed to guide MediHaven, and is outlining the user roles, anticipated system behaviour, input constraints, and the checks applied during the development, to a significant degree; in fact, the verification status in the tables refers to the tests and demonstrations conducted for this project; it is not a certification of clinical suitability or production security, in actual application.

3.2 User Classes, Personas and Operational Roles

MediHaven supports four separate user roles, enforced via server-side session authentication:

● **Attending Physician / Triage Nurse, such as Need**s real-time access to ward dashboards, colour-coded patient cards, vital sign trajectory charts, multi-model risk scores, decision tree physical rule breadcrumbs, and instant deterioration alert banners, in actual application.
● Can create time-limited QR access passes with custom category selections, configure usage quotas, and cancel active passes at will, broadly speaking. Along these dimensions, access to personal vault records sorted into the vitals, allergies, medications, diagnoses, surgeries, notes and labs is needed by patient / Health Sovereign: in many respects.
● **Clinical Researcher, namely Need**s access to cohort-level analytics, population risk distribution graphs, feature correlation matrices, and the interactive what-if simulation sandbox to test counterfactual physiological hypotheses, to a significant degree.
● **System Administrator, namely** Monitors application health endpoints, database connection states, model memory telemetry, audit log activity, and background worker queues, in actual application.

3.3 Functional Requirements

Table 3.1 lists the functional needs and the matching execution or test module.

Table 3.1: Functional is needing specification and implementation status

Req ID	Requirement Description	Input / Trigger Condition	System Output & Behaviour	Verification Status
FR-01	Public View & Navigation	GET request to /	Renders responsive landing page with platform overview	Verified (test_presentatio n. py)
FR-02	Role-Based Authentication	POST /api/auth/login	Authenticates user against secure session and designates RBAC role	Verified (test_auth.py)

Req ID	Requirement Description	Input / Trigger Condition	System Output & Behaviour	Verification Status
FR-03	Patient Roster Retrieval	GET /api/patients	List of patients with demographics and risk scores was, strictly speaking, filtered by returns	Verified (test_api.py)
FR-04	Data Cleaning & Ingestion	Database load event	Validates vitals/labs against range constraints and clamps outliers	Verified (test_data_pipeline.py)
FR-05	30-Feature Engineering	Feature pipeline call	Pulls demographics, latest vitals, clusters, OLS slopes, deltas	Verified (test_data_pipeline.py)
FR-06	Multi-Model Ensemble	POST /api/predict	Runs 4 models, implements 0.30/0.30/0.25/0.15 weights, returns tier and score	Verified (test_models.py)
FR-07	Deterioration Alerting	Inference threshold breach	Triggers HIGH/CRITICAL alert banners and real-time dashboard updates	Verified (test_api.py)
FR-08	Scoped QR Generation	POST /api/vault/tokens	Signs HMAC-SHA256 JWT, saves metadata, returns Base64 QR code	Verified (test_vault.py)
FR-09	Zero-Trust Scope Check	POST /api/vault/verify	Validates signature, expiry, quota; returns only permitted categories	Verified (test_vault.py)
FR-10	Tamper-Evident Audit	Any QR scan attempt	Logs timestamp, token ID, patient ID, status (GRANTED/DENIED/EXPIRED)	Verified (test_vault.py)
FR-11	Clinical Narrative Copilot	GET /api/narrative/<id>	Summary or deterministic local text were blended by Returns Gemini AI	Verified (test_nlp.py)
FR-12	What-If Risk Simulation	POST /api/research/whatif	Recalculates multi-model predictions on altered virtual factors	Verified (test_research_mode_app.py)

3.4 Non-Functional Needs

Table 3.2 lists the non-functional needs and the checks applied to review them in the development environment, in real-world scenarios.

Table 3.2: Non-functional needs and engineering verification

Category	Non-Functional Need	Target Metric	Observed Achievement
Inference Latency	Model prediction pipeline must not block clinical triage	< 50ms per patient	18.62ms average latency gathered
Page Load Speed	Dashboard views must, in essence, render swiftly on hospital workstations	< 500ms initial load	Rendered in < 120ms via local SQLite
Cryptographic Rigor	Token signatures must, by design, prevent forgery and unauthorized tampering	HMAC-SHA256 (256-bit key)	All altered payloads in the test set were rejected
Database Concurrency	Concurrent reads must not block background observation writes	Zero read lockouts	WAL mode allowed; concurrent reads and one writer are normally supported
Software Reliability	External API downtime must never crash core triage operations	100% graceful fallback	Fallback test passed when the external service was unavailable
Test Suite Quality	Across all modules, broad automated regression coverage	> 85% code coverage	366 tests passed across 14 modules; code coverage percentage not reported

3.5 System Constraints, Hardware and Software Dependencies

MediHaven is intended to run on a standard computer. Building on this observation, the minimum configuration applied for planning was a dual-core 2.0 GHz x86-64 processor, 4 GB RAM, and 500 MB of free storage, on the whole. The interface was inspected in current desktop versions of Chrome, Firefox, Edge and Safari, in many respects. The software dependencies are listed in needs.txt; the versions recorded for this project include Python 3.10+, Flask 3.1.3, Werkzeug 3.1.9, pandas 3.0.6, NumPy 2.5.4, scikit-learn 1.9.1, PyJWT 2.15.1, qrcode 8.2, Pillow 12.3.0, opencv-python-headless 5.0.0.93, and pyzbar 0.1.9.

3.6 Detailed Use Case Models

The following use cases outline the principal user interactions supported by the prototype:

- **UC-01 (Physician Ward Triage): A physician signs in, opens the dashboard, reviews the patient list, and designates a patient to view recent remarks, risk output, and the available Decision Tree rule path, in many respects.**
- **UC-02 (Model Inference): an inference request is received by The framework, retrieves the patient findings, creates the 30-feature vector, runs the four model components, and stores the produced scores in the predictions table, in actual application.**

- **UC-03 (Patient QR Pass Generation): A patient opens the vault, picks the categories to share (as an illustration, vitals and allergies), sets an expiry time and usage limit, and yields a QR pass.**
- **The server is verifying the HMAC, expiry, revocation state, usage limit, and requested category before returning the permitted data and writing an audit entry, on the whole; indeed, UC-04 (QR Pass Scanning and Audit): A clinician is scanning the QR code.**
- **The values are hypothetical and are not implemented to the patient record. UC-05 (What-If Research Simulation): A researcher opens a patient profile in the research view, changes picked input values, and reruns the model pipeline for the comparison of the consequent scores, broadly speaking.**

Chapter 4: System Architecture and Design

4.1 Overall Architectural Design and Module Decomposition

A modular monolithic architecture is applied by MediHaven. Taken together, its Python packages separate API routes, data processing, model inference, the medical vault, and database operations, in real-world scenarios; in the meantime, the modules run, within one Flask application; this was a concrete choice for a semester project, because it is avoiding the extra rollout and communication work needed by separate microservices, to a significant degree.

Figure 4.1 is revealing how requests move from the browser to the Flask application and then to the data, model, database and vault modules, to a significant degree.

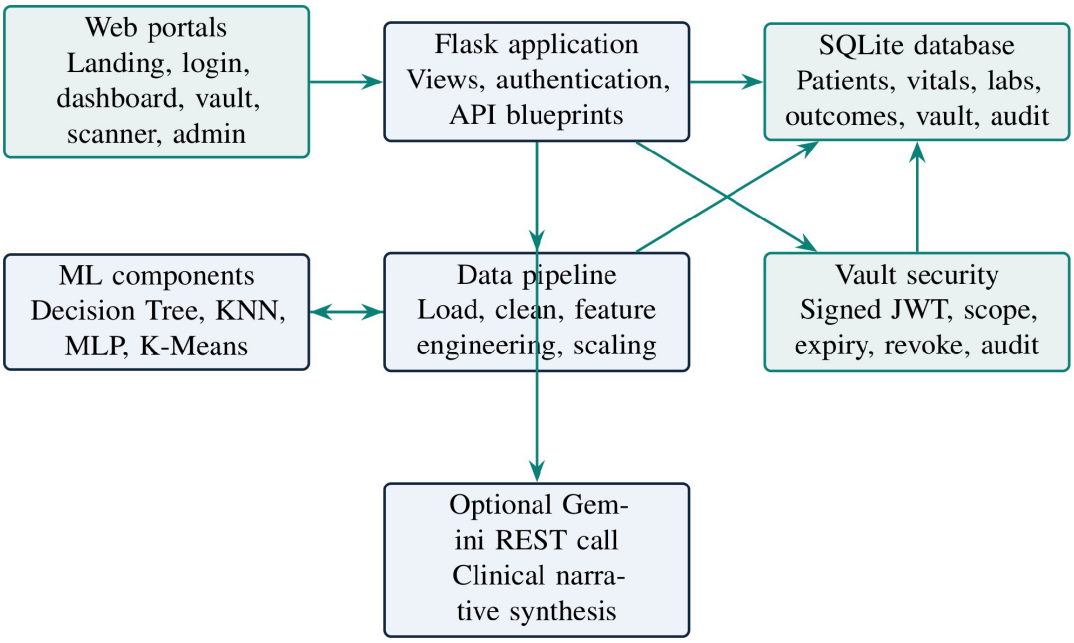


Figure 4.1: MediHaven modular system architecture

Table 4.1 is outlining the primary engineering packages that include MediHaven, along with their core responsibilities and connections, broadly speaking.

Table 4.1: Core software implementation modules and reliances

Package / Directory	Core Architectural Responsibility	Key Internal & External Dependencies
src/api/	Flask application factory, HTTP route blueprints, session RBAC, and error handlers	Flask, Flask-CORS, src/data, src/models, src/vault
src/data/	Data loading, physiological cleaning, 30-feature engineering, and normalization	pandas, numpy, scipy, src/database

Package / Directory	Core Architectural Responsibility	Key Internal & External Dependencies
src/models/	Model wrappers (Decision Tree, KNN, MLP, K-Means), ensemble fusion, explainability	scikit-learn, joblib, src/data
src/vault/	HMAC-SHA256 JWT creation, QR encoding/decoding, scope checking, and audit logging	PyJWT, qrcode, pillow, opencv, pyzbar, src/database
src/database/	SQLite connection manager, schema initialization, and 100-patient synthetic seeder	sqlite3, src/utils
src/nlp/	Clinical narrative copilot integration (Gemini REST client with deterministic fallback)	requests, json, src/utils
src/training/	MIMIC-IV research dataset loaders, leakage-free splits, and model training pipelines	duckdb, scikit-learn, pyarrow
templates/ & static/	Server-rendered HTML5 views, clinical CSS design system, and vanilla JS controllers	Jinja2, Chart.js, HTML5 Canvas
tests/	Broad automated pytest suite (unit, integration, security and benchmark tests)	pytest, unittest.mock, tempfile

4.2 End-to-End Clinical Data Pipeline Flow

Figure 4.2 is revealing the data flow employed by the application. Patient records and time-stamped notes are read from SQLite. The data-cleaning step is checking values against the ranges configured for the project and handles missing remarks as stated by the pipeline rules, on the whole. The 30 input features are then created by the feature pipeline and implements the scaler fitted during the training before passing the data to the model components, in actual application.

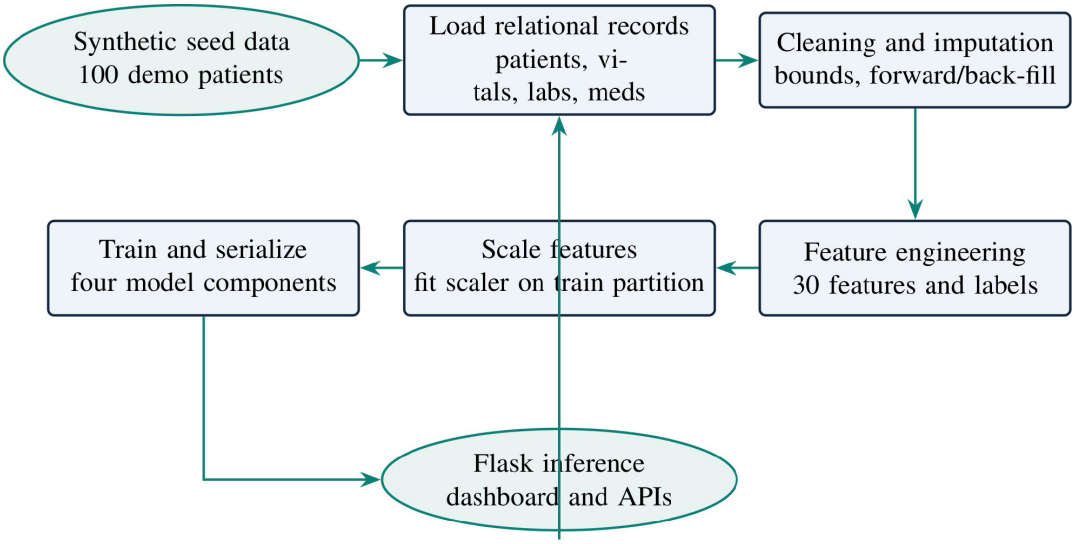


Figure 4.2: End-to-end data pipeline flow from raw storage to inference

4.3 Unified Modeling Language (UML) Behavioral Models

The four diagrams within the current section outline the central system interactions, namely the user use cases (Figure 4.3), module relationships (Figure 4.4), QR pass creation and verification (Figure 4.5), and model execution flow (Figure 4.6), to a significant degree.

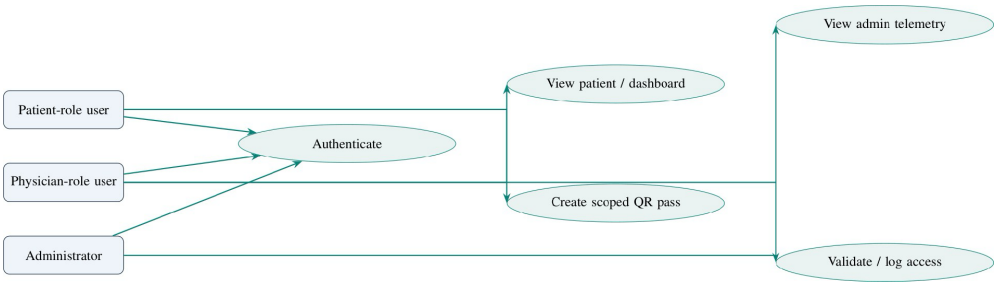


Figure 4.3: Use-case behavioral diagram across the user roles and workflows

A widespread calling pattern is applied by the model components so that they can be invoked by the ensemble without placing model-specific code inside the API routes, in real-world scenarios. Figure 4.4 is revealing the relationships between the Flask route controllers, the data-loading and feature-processing classes, the ensemble predictor, and the vault service, broadly speaking.

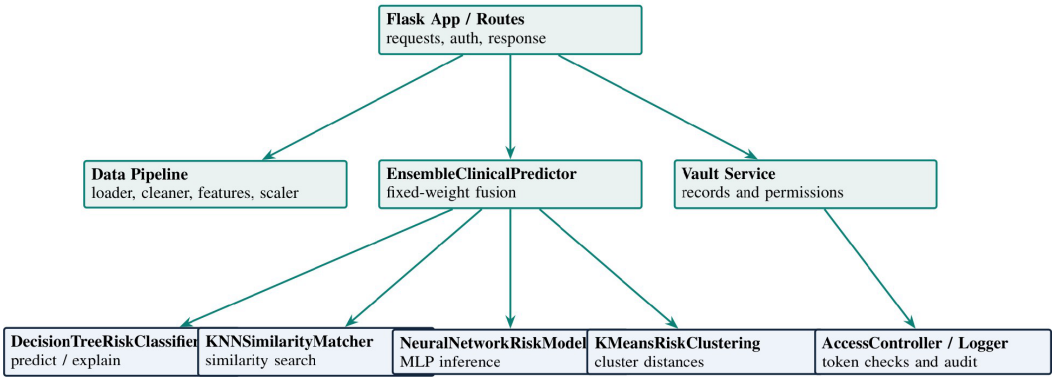


Figure 4.4: Part and class dependency diagram

Figure 4.5 is revealing the QR pass workflow. When a patient creates a pass, the server is building a JWT incorporating the permitted categories and token metadata, validates it with HMAC-SHA256, and stores the associated token state in SQLite, and checks the token and its database record before returning any medical data, in actual application. In this broader context, the access attempt is then written to the audit log.

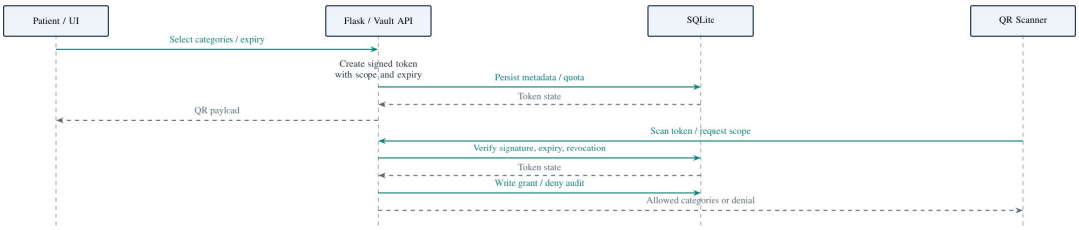


Figure 4.5: Cryptographic QR vault token creation and validation sequence

Figure 4.6 is revealing the four model components applied in the current ensemble. Simultaneously, k-Means contributes a score grounded in distance to cluster centroids; also model outputs are, by definition, returned by the Decision Tree, MLP, and KNN components, in real-world scenarios. The application combines these component outputs applying the configured weights to yield a risk tier and an overall score, in many respects. The cluster-derived value is a similarity score and should not be construed as a optimized class probability, in real-world scenarios.

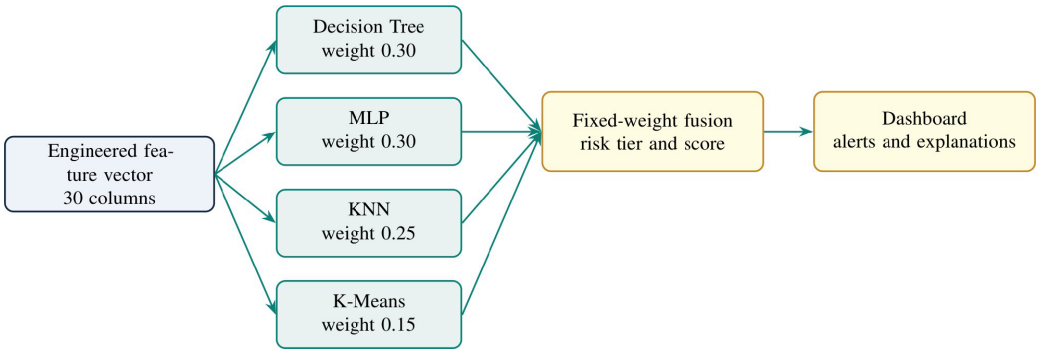


Figure 4.6: Four-component ensemble fusion and scoring flow

4.4 Relational Database Design and Entity Integrity

SQLite is applied for storage, with Write-Ahead Logging (WAL) permitted, and still, in the meantime, changes to the WAL file are appended by a writer, on the whole; and thus, WAL is allowing readers to continue reading a uniform database snapshot. Against this backdrop, also, sQLite still packs write transactions, so the configuration does not supply unlimited concurrent writes, to a significant degree; nine tables are encompassed by the database, with relationships and restrictions laid out in Figure 4.7.

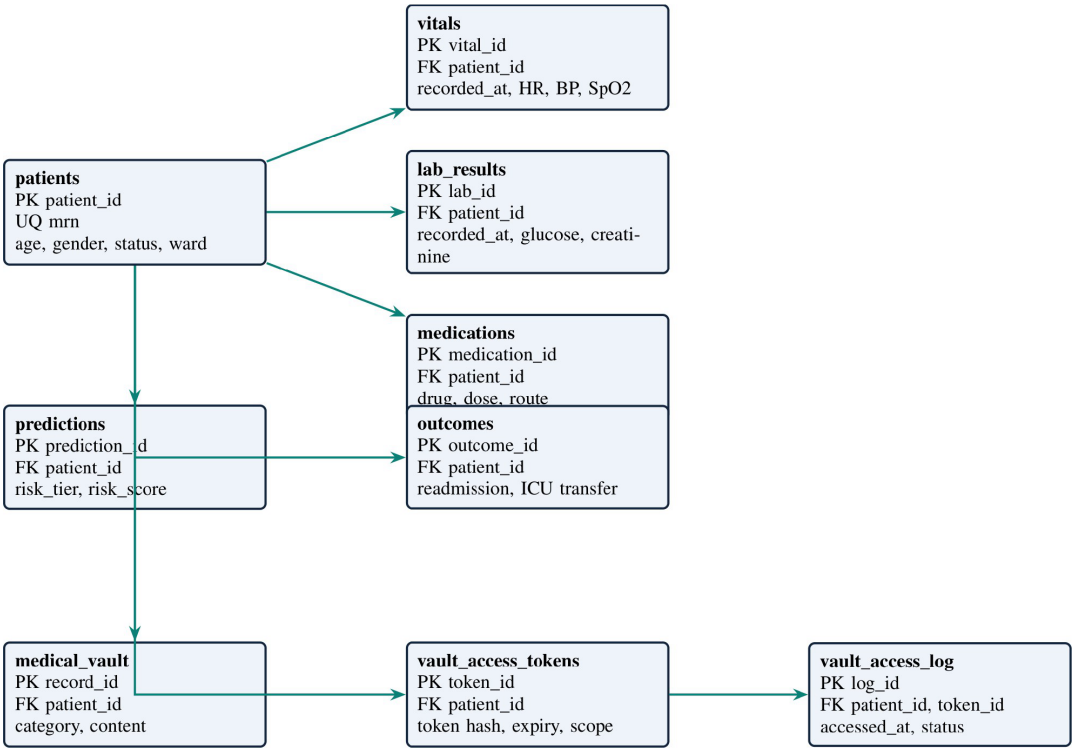


Figure 4.7: Relational database Entity-Relationship (ER) schema diagram

The nine tables applied by the current database schema are outlined below:

- Fields: patient_id (PK), mrn (UNIQUE), full_name, age, gender, admission_date, status (ACTIVE/DISCHARGED/TRANSFERRED), ward (ICU/CARDIAC/GENERAL/SURGICAL), attending_physician, so patients (Parent Entity): Stores primary demographic and administrative state, in actual application.
- **vitals (Time-Series):** Stores longitudinal vital sign measurements. Composite index on (patient_id, recorded_at). Viewed through this lens, fields, including vital_id (PK), patient_id (FK), recorded_at (TIMESTAMP), heart_rate (CHECK 20-300), systolic_bp (CHECK 40-300), diastolic_bp (CHECK 20-200), temperature (CHECK 30-45 C), respiratory_rate (CHECK 4-80), oxygen_saturation (CHECK 50-100), on the whole.
- Fields, including lab_id (PK), patient_id (FK), recorded_at (TIMESTAMP), glucose (CHECK 10-1000 mg/dL), creatinine (CHECK 0.1-25 mg/dL), blood_urea_nitrogen (CHECK 1-200 mg/dL), white_blood_cells (CHECK 0.5-150 K/uL), hemoglobin (CHECK 2-25 g/dL), platelets (CHECK 10-2000 K/uL), in real-world scenarios. From this perspective, lab_results (Time-Series): Stores biochemical laboratory panels.
- Fields, including medication_id (PK), patient_id (FK), drug_name, dosage, route (IV/ORAL/SC/IM), frequency, start_date, end_date, is_active (BOOLEAN), in actual application. Medications (Prescription Entity): Monitors ran pharmaceuticals.
- Fields, such as prediction_id (PK), patient_id (FK), evaluation_time, risk_tier (LOW/MEDIUM/HIGH/CRITICAL), risk_score (0.0-1.0), readmission_score (0.0-1.0), model_outputs (JSON), rules_fired (JSON), on the whole. Predictions (Model Output Entity): Records historical deterioration risk evaluations with this parallely.
- Fields like - outcome_id (PK), patient_id (FK), discharge_status, readmitted_30d (BOOLEAN), icu_transfer (BOOLEAN), length_of_stay_days, in_hospital_mortality (BOOLEAN), which is normally why outcomes (Clinical Ground Truth): Stores gathered clinical endpoints, on the whole.
- Fields also are encompassing record_id (PK), patient_id (FK), category, title, content, record_date, and metadata_json, in actual application; in fact, within the mentioned framework, medical_vault (Categorized Patient Data): Stores patient records in seven diverse categories, including vitals, allergies, medications, diagnoses, surgeries, notes and labs, in real-world scenarios.
- vault_access_tokens (Token Metadata): Sustains token state and authorization restrictions; truly, fields, spanning token_id (PK), patient_id (FK), token_hash (UNIQUE), scope (JSON array of permitted categories), issued_at, expires_at, max_uses, use_count, is_revoked (BOOLEAN), to one notable degree, in many respects.
- vault_access_log (Access Audit Log): Stores access attempts (logs), spanning log_id (PK), patient_id (FK), token_id (FK), accessed_at, requested_category, access_status, accessor_ip, and user_agent, in actual application; any entry for each individual individual is predominantly stored in this application by the application handled access attempt, broadly speaking.

4.5 Cryptographic Vault Architecture and Zero-Trust Protocols

The vault is, in effect, using a server-side access check grounded in the token scope. HMAC-SHA256 verifies that the JWT payload has not been changed by a party without the shared secret; this check does not, by itself, restrict which entries can be returned, on the whole; after verification, the application checks expiry, revocation, usage count, patient identifier, and the requested category, to a significant degree. In the SQL query, it then encompasses this permitted category or categories, which is why a request for a category outside the token scope is denied; the finding of the attempt is recorded by the audit table. The scope check is therefore executed by the application and database query, not by the JWT format alone.

Chapter 5: Algorithm Design and Data Structures

5.1 Critical Computational Workflows

The primary processing tasks in MediHaven are the feature extraction pipeline, the model ensemble, in actual application; indeed, also, the QR token validation procedure; those tasks are, in essence, outlined separately, because each has distinct inputs, outputs and validation requirements, to a significant degree.

5.2 Feature Engineering Algorithm and Longitudinal Slope Extraction

Any single vital-sign reading does not reveal whether a value has been stable or changing, in real-world scenarios; on this account, the feature pipeline also is evaluating the slope of picked remarks over a configurable 72-hour window, to a significant degree. In this broader context, the slope is estimated applying Ordinary Least Squares (OLS) regression; it outlines the direction and rate of change in the available measurements; it is not, by itself, a clinical diagnosis.

For a series of timestamps t_i (in hours relative to the most recent measurement) and gathered values y_i, the OLS trajectory slope beta is computed applying closed-form analytical regression:

```
beta = [ Sum( (t_i - t_bar) * (y_i - y_bar) ) ] / [ Sum( (t_i - t_bar)^2 ) ]
```

The feature pipeline returns thirty quantitative columns in what follows order:

1-3. Demographics, namely age, gender_male, gender_female.

4-8. In parallel with this, latest Vital Observations, like heart_rate_last, systolic_bp_last, diastolic_bp_last, temp_c_last, spo2_last.

9-16. In light of this dynamic, 72-Hour Longitudinal Summary Aggregates - heart_rate_mean, heart_rate_min, heart_rate_max, heart_rate_std, systolic_bp_mean, systolic_bp_min, systolic_bp_max, spo2_min.

17-19. Against this backdrop, physiological Trajectory Slopes, like heart_rate_slope (bpm/hr), systolic_bp_slope (mmHg/hr), spo2_slope (%/hr).

20-22. Taken together, formulated Hemodynamic Indicators, such as mean_arterial_pressure = (systolic_bp_last + 2 * diastolic_bp_last) / 3; pulse_pressure = systolic_bp_last - diastolic_bp_last; shock_index = heart_rate_last / systolic_bp_last, in many respects.

23-27. Against this backdrop, latest Laboratory Panels, such as glucose_last, creatinine_last, bun_last, wbc_last, hemoglobin_last.

28-29. Under these conditions, longitudinal Laboratory Deltas, like glucose_delta (first-to-last), creatinine_delta (first-to-last).

30. Active Medication Burden: medication_count (count of concurrently recommended active drugs).

The formal feature engineering pseudocode is detailed below:

```
ALGORITHM, namely ExtractClinicalFeatureVector
INPUT: patient_id, database_connection
OUTPUT: feature_vector of length 30
BEGIN
  patient = SELECT * FROM patients WHERE patient_id = patient_id;
  vitals = SELECT * FROM vitals WHERE patient_id = patient_id ORDER BY recorded_at ASC;
  labs = SELECT * FROM lab_results WHERE patient_id = patient_id ORDER BY recorded_at ASC;
  meds = SELECT COUNT(*) FROM medications WHERE patient_id = patient_id AND is_active = 1;
  latest_vital = vitals.last_row();
  window_vitals = vitals.filter(recorded_at >= latest_vital.recorded_at - 72_HOURS);

  hr_slope = ComputeOLSSlope(window_vitals.timestamps, window_vitals.heart_rate);
  bp_slope = ComputeOLSSlope(window_vitals.timestamps, window_vitals.systolic_bp);
  spo2_slope = ComputeOLSSlope(window_vitals.timestamps, window_vitals.oxygen_saturation);

  map_val = (latest_vital.systolic_bp + 2 * latest_vital.diastolic_bp) / 3.0;
  pulse_pressure = latest_vital.systolic_bp - latest_vital.diastolic_bp;
  shock_index = latest_vital.heart_rate / MAX(latest_vital.systolic_bp, 1.0);

  Assemble 30-dimensional array in strict FEATURE_COLUMNS order;
  Impute absent characteristics with physiological reference defaults;
  RETURN feature_vector;
END
```

5.3 Multi-Model Inference and Weighted Ensemble Fusion

Rather than counting on a single model architecture, MediHaven executes an ensemble of four mixed algorithms. Under these conditions, each algorithm contributes a unique viewpoint on the patient's physiological state:

- **Decision Tree Classifier (Weight = 0.30):** The tree is trained on unscaled attributes with one further maximum depth of 5, to a significant degree; in fact, its split thresholds are remaining in the original feature units, which is allowing the application to show rule paths such as the threshold on shock index or glucose, broadly speaking. Taken together, all these rules outline the trained model; they should not be treated as medical guidelines.

- **The implementation is using two output heads, including one for the deterioration-risk classes and one for the 30-day readmission estimate, broadly speaking. In light of this dynamic, multilayer Perceptron (Weight = 0.30):** The network is using hidden layers of sizes 64 and 32 with ReLU activations and the Adam optimizer, in actual application.
- **K-Nearest Neighbors (Weight = 0.25):** The KNN component uses K=5 and Euclidean distance on standardized traits; it returns the five closest data in the reference cohort, giving the interface examples of entries with akin feature values, to a significant degree.
- **K-Means Clustering (Weight = 0.15):** K=4 centroids are executed by The clustering component; it is evaluating the distance from the patient feature vector to each individual centroid and converts the distances into a normalized cluster-similarity score; this score is not a class probability unless a separate, recorded mapping from clusters to outcome classes is established, since K-Means is unsupervised, on the whole.

The current deployment combines the component outputs applying fixed weights configured for this project:

P_ensemble(class_j) = 0.30 * P_DT(j) + 0.30 * P_MLP(j) + 0.25 * P_KNN(j) + 0.15 * P_KMeans(j)

This architecture designates the risk tier associated with the highest combined class score; in turn, this showed scalar score is bounded by the deployment between 0 and 1, in many respects; because, the ensemble is encompassing a cluster-similarity insight and the demonstration cohort is synthetic, the score should not be outlined as a clinically optimized probability, to a significant degree.

5.4 Cryptographic QR Token Generation and Validation Algorithm

The medical vault enforces sovereign access through an end-to-end cryptographic protocol:

```
ALGORITHM, namely ValidateQRVaultAccess
INPUT: raw_qr_jwt_string, requested_category, accessor_metadata
OUTPUT: access_decision (GRANTED / DENIED), authorized_records_payload
BEGIN
  claims = DecodeAndVerifyHMACSHA256(raw_qr_jwt_string, SECRET_KEY);
  IF claims IS INVALID OR SIGNATURE_MISMATCH THEN
    LogAudit(token_id=NULL, status='DENIED', reason='INVALID_SIGNATURE');
    RETURN (DENIED, 'Cryptographic signature verification failed');
  END IF

  IF CURRENT_TIMESTAMP() > claims.expires_at THEN
    LogAudit(claims.token_id, status='EXPIRED', reason='TOKEN_PAST_EXPIRY');
    RETURN (DENIED, 'Access pass has expired');
```

```
  END IF

  token_record = SELECT * FROM vault_access_tokens WHERE token_id = claims.token_id;
  IF token_record.is_revoked == 1 THEN
    LogAudit(claims.token_id, status='REVOKED', reason='TOKEN_MANUALLY_REVOKED');
    RETURN (DENIED, 'Access pass has been revoked by patient');
  END IF

  IF token_record.use_count >= token_record.max_uses THEN
    LogAudit(claims.token_id, status='EXPIRED', reason='MAX_USES_EXCEEDED');
    RETURN (DENIED, 'Maximum permitted scans reached');
  END IF

  IF requested_category NOT IN token_record.scope THEN
    LogAudit(claims.token_id, status='SCOPE_MISMATCH', reason='CATEGORY_UNAUTHORIZED');
    RETURN (DENIED, 'Requested category outside permitted scope');
  END IF

  UPDATE vault_access_tokens SET use_count = use_count + 1 WHERE token_id = claims.token_id;
  records = SELECT * FROM medical_vault WHERE patient_id = claims.patient_id AND category =
requested_category;
  LogAudit(claims.token_id, status='GRANTED', reason='AUTHORIZED_ACCESS');
  RETURN (GRANTED, records);
END
```

5.5 Computational Time and Space Complexity Analysis

Let us consider n to represent the number of longitudinal findings per patient (commonly 50-200), d = 30 the feature dimensionality, N = 100 the number of historical reference patients, K = 4 the cluster count, and h <= 5 the Decision Tree depth, in many aspects, table 5.1 details the asymptotic time and space complexity of each primary operation in MediHaven, on the whole.

Table 5.1: Asymptotic computational time complexity of core algorithms

Algorithmic Operation	Time Complexity	Space Complexity	Operational Notes
Longitudinal Observation Sorting	O(n log n)	O(n)	Timsort on timestamp index; ordinarily < 0.2ms per patient
72-Hour Window Aggregation	O(n)	O(1)	Single linear scan over chronologically bounded rows
OLS Trajectory Slope Calculation	O(n)	O(1)	Closed-form sum accumulation; zero iterative tuning

Algorithmic Operation	Time Complexity	Space Complexity	Operational Notes
Feature Vector Standardization	O(d)	O(d)	Vectorized element-wise multiply-add: (x - mu) / sigma
Decision Tree Inference	O(h) <= O(5)	O(tree nodes)	Binary threshold traversal; completed in < 0.1ms
MLP Forward Propagation	O(d*h1 + h1*h2 + h2*c)	O(weights)	Matrix multiplication across the layer dimensions (30x64 + 64x32 + 32x4)
KNN Precedent Search	O(N * d + N log k)	O(N * d)	Exact Euclidean distance matrix scan over stored reference cohort
K-Means Centroid Distance	O(K * d)	O(K * d)	Distance to 4 centroids in 30-dimensional space
Ensemble Probability Fusion	O(c) = O(4)	O(c)	Linear scalar combination across the four class probabilities
HMAC-SHA256 Token Validation	O(m)	O(m)	Cryptographic hash over token string of length m (~250 bytes)
SQLite Indexed Query	O(log M + r)	O(r)	B-tree index search over M table rows returning r result rows

5.6 Selection and Justification of Data Structures

The selection of data structures was driven by strict computational and security standards:

- Taken together, slicing windowed vitals and assessing statistical moments is being ran in gathered C extensions, to avoiding slow Python construed loops, in actual application. Under these conditions, **pandas DataFrames & NumPy ndarrays**, incorporating Applied, throughout src/data/ for high-performance vectorized quantitative operations, on the whole.
- **B-Tree Database Indexes, such as SQLite composite B-tree indexes** (patient_id, recorded_at) and (token_hash) are reducing time-series and token retrieval from O(M) full-table scans to O(log M) logarithmic traversals, to a significant degree.
- **Layered Binary Tree (scikit-learn):** Encapsulates Decision Tree split thresholds, allowing instantaneous O(h) check and clean traversal for rule generation, in many respects.
- **JSON Web Token Claims Dictionary: Offers a compact, URL-safe, HMAC-authenticated data structure for transient QR encoding without needing server round-trips to the server for foundational metadata, in many respects.**

Chapter 6: Subject-Wise Engineering Concepts & APNI Leap Alignment

6.1 Software Engineering Principles

The application is following the modular architecture, such as route handlers in src/api/ handle HTTP requests and authorization, in many respects; truly, the pytest suite is implemented to check behaviour as modules are changed, which is why in the interim, data processing and prediction logic are ran in src/data/ and src/models/, on the whole. In this broader setting, chapter 9 is, put simply, listing the test modules and the primary cases covered by them, in real-world scenarios.

6.2 Computer Networks and RESTful Communication

The browser communicates with the Flask backend over HTTP applying JSON requests. Before processing JSON payloads, the anticipated request format and content type is authenticated by the API, on the whole. Flask-CORS is configured to control cross-origin access. Along these dimensions, gET requests retrieve information, POST requests submit prediction or token-generation requests, and DELETE requests revoke tokens, in real-world scenarios.

6.3 Database Management Systems and Relational Normalization

Relatively stable patient information from repeated findings is separated by the database; in turn, also, patient demographics are stored in the patients table, on the whole. During this period, time-stamped vital signs and laboratory results are stored in their own tables and linked by foreign keys, in many respects. Within this framework, simultaneously, changes to the WAL file are appended by a writer, so also, sQLite WAL mode is allowing readers to continue applying a uniform snapshot. Write transactions are still serialized, and checkpointing is needed to move committed changes from the WAL file back into the primary database.

6.4 Operating Systems and Resource Management

MediHaven is running as a Python application process; and thus, the application can (in practical terms) be served through a WSGI server such as Waitress or Gunicorn; the operating system schedules the worker threads employed by the server, on the whole. Within this framework, database connections are managed through the project's connection utilities, and model files are loaded when the application starts, in actual application, and environment variables are applied for configuration values such as the database path and token secret.

6.5 Data Structures and Investigation of Algorithms

The project is using arrays and DataFrames for quantitative processing, tree structures for Decision Tree inference, and Euclidean distance calculations for KNN and K-Means, broadly speaking; HMAC-SHA256 is employed to corroborate QR token contents. From this perspective, all these examples are connecting the data structures and algorithm concepts inspected in the course to precise parts of the application, broadly speaking.

6.6 Academic Subject-to-Implementation Mapping Matrix

Table 6.1 maps undergraduate computer engineering subjects to their exact deployment in MediHaven.

Table 6.1: Undergraduate curriculum subject-to-implementation mapping matrix

Undergraduate Course / Subject	Academic Engineering Concept	Direct Execution in MediHaven Codebase
Software Engineering (SE)	Architectural patterns, separation of concerns, unit testing	Modular monolith (src/api, src/data, src/models), 366 pytest cases
Database Management Systems (DBMS)	Relational modeling, 3NF, B-tree indexes, ACID, WAL	9 normalized tables in SQLite, foreign keys, CHECK constraints, WAL mode
Computer Networks (CN)	Client-server architecture, HTTP/REST, JSON schemas, CORS	Flask RESTful route blueprints, fetch() controllers, JWT payload transfer
Operating Systems (OS)	Process execution, I/O management, concurrency, environment variables	Python process model, multi-threaded Waitress WSGI, dotenv configuration
Data Structures & Algorithms (DSA)	Asymptotic intricacy, sorting, trees, distance metrics, hashing	OLS slope regression, Decision Tree binary splits, KNN distance, SHA-256
Information & Network Security (INS)	Message authentication, token authorization, scope checks, audit logging	HMAC-SHA256 JWT signing, QR encoding, scope isolation, fixed logging
Machine Learning (ML)	Supervised grouping, regression, clustering, ensemble techniques	Decision Tree, KNN (K=5), MLP Neural Network, K-Means (K=4), 30-feature vector

6.7 Alignment with Sardar Patel Institute of Technology APNI Leap

Ideas from several computer engineering subjects are implemented by MediHaven in one project, and building and blending the parts offered any concrete way to employ topics from the Semester V curriculum, on the whole. Viewed through this lens, the work is encompassing a further

relational database, a web API, data processing, machine-learning models, QR-based access control, and automated tests, broadly speaking, and the prototype is not one production hospital system, so yet, it is revealing how these subjects can (in practice) contribute to a larger software application.

Chapter 7: Execution and Module-Wise Coding

7.1 Development Environment and Toolchain

The project was devised applying Python 3.10.4 on Windows 11 and Linux; flask 3.1.3 was applied for the web application, Waitress 3.0.2 for the WSGI server, and pytest 9.1.1 for automated testing, in real-world scenarios. Under these conditions, connections were installed in a project-specific virtual environment (.venv), so the team applied IDE-based development and flake8 for code-style checks, in many respects.

7.2 Phased 8-Stage Development Methodology

Development was divided into the eight phases. In parallel with this, the phases outline the principal modules that were operationalized and brought together during the semester:

Phase 1. Project Setup: Created the directory structure, logging and configuration utilities, CSS design tokens, and base HTML templates.

Phase 2. Database and Demonstration Data, including Created the nine-table SQLite schema, allowed WAL mode, and wrote a seeder that is yielding a 100-patient mock data with vital signs, laboratory results, medications and outcomes, on the whole.

Phase 3. Against this backdrop, data Processing and Feature Extraction: Implemented the data loader, range checks, implemented a 30 feature extraction pipeline, and scaler applied by the model workflow, in many respects; in fact, the training pipeline is keeping preprocessing within the training split.

Phase 4. Along these dimensions, machine-Learning Models - Implemented 4 models which include - Decision Tree, KNN, MLP, and K-Means parts, along with the ensemble module that is combining their outputs, to a significant degree; in fact, the model artifacts are saved under the models_store/.

Phase 5. Within this framework, medical Vault and QR Passes: Executed the vault service, HMAC-authenticated JWT generation, QR scanning, category-scope checks, token state checks, and access logging, in actual application.

Phase 6. Along these dimensions, flask API and Authentication: Implemented the Flask application factory, session-based role checks, input validation, and route blueprints for patients, predictions, alerts, vault functions, and research functions, in actual application.

Phase 7. In parallel with this, browser Interface: The triage dashboard has been built also, patient vault, QR scanner, and administration views applying Jinja2 templates, CSS and JavaScript.

Phase 8. Within this framework, testing and Demo Launcher, such as Added the benchmark script, also completed the test suite recorded in Chapter 9, and there exists run_demo.py to startuply check and launch the application, in real-world scenarios.

7.3 Core Engineering Modules Deep Dive

This section outlines the primary implementation modules and the part each one plays in the application:

- **Feature Engineering (src/data/feature_engineering.py): The extract_feature_vector() function is reading patient findings from pandas DataFrames, checks the time range, assesses OLS slopes over the 72-hour window, determines MAP, pulse pressure, and shock index, and returns the 30-feature representation, on the whole; in fact, taken together, missing values are handled applying the defaults configured in the pipeline.**
- **The application is using the tree structure and threshold values to reveal the decision path followed for the input record, in many respects; Decision Tree (src/models/decision_tree_model.py): The model is trained on the unscaled feature representation.**
- **Ensemble Predictor (src/models/ensemble.py): This module loads the saved model artifacts, prepares the input features, calls the four model components, and combines their outputs applying the configured weights; it returns any risk tier and score, together with supporting model information, on the whole; in actual application. Also, thus, this score is one prototype output and is not offered as a further clinically aligned probability, in actual application.**
- **QR Token Module (src/vault/qr_generator.py and qr_scanner.py): The generator is creating a JWT incorporating token metadata such as subject, permitted scopes, expiry and usage limit, in real-world scenarios; the scanner is using OpenCV and pyzbar to decode QR codes from image files or camera input, to a significant degree. PyJWT is validating the token employing HMAC-SHA256. Also, the QR image is created by the qrcode package.**

7.4 Front-End User Interface Design System and Application Screenshots

The interface is using a dark-slate colour palette specified, through CSS custom properties in static/css/tokens.css, on the whole. The page background, card surfaces, borders, accent colour, and colours employed for the four risk tiers are, by design, stated by the variables, in actual application. Framework sans-serif fonts are employed by the pages and share typical styles for buttons, cards and navigation, broadly speaking.

The following figures are notably screenshots encapsulated from the application running in the local development environment:

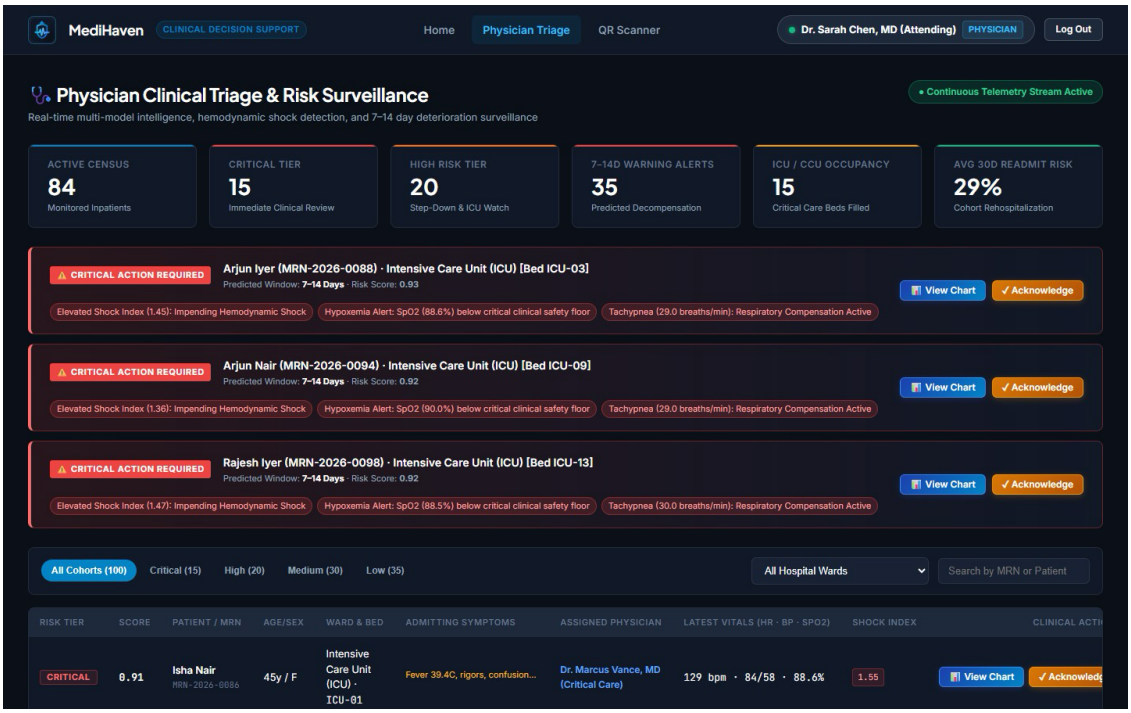


Figure 7.1: MediHaven Triage Dashboard with colour-coded risk cards, vital sign trend sparklines, and active alert banners

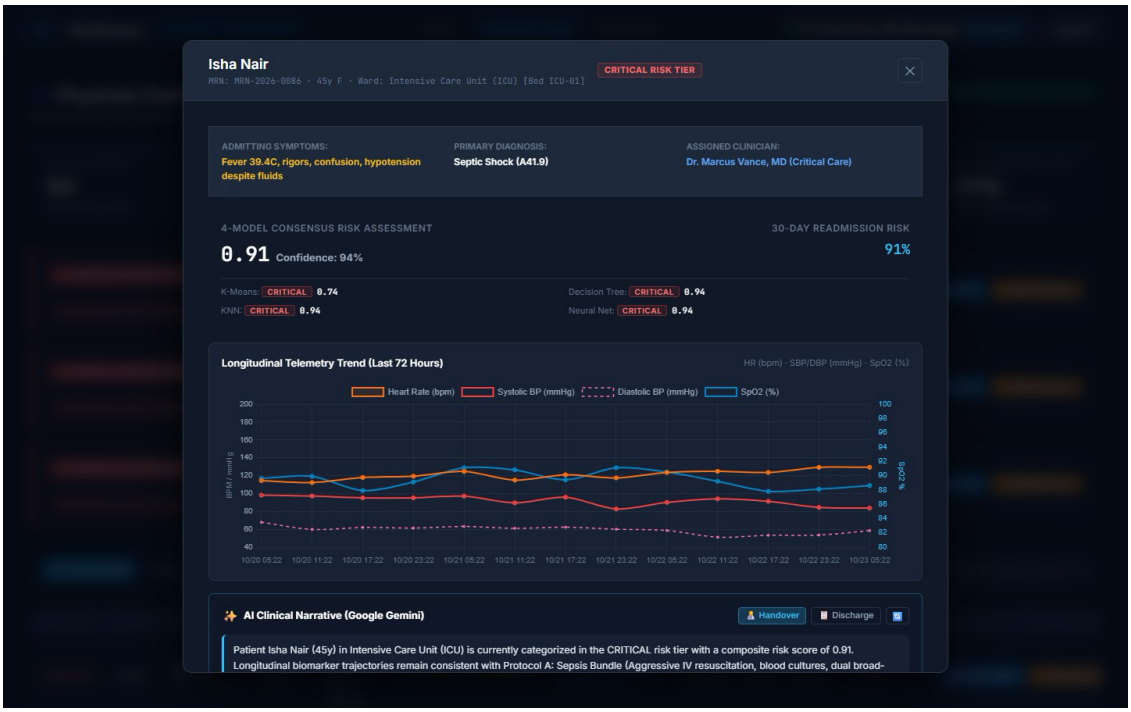


Figure 7.2: Patient Chart Drawer revealing multi-model consensus, Decision Tree physical rules, and KNN historical matches

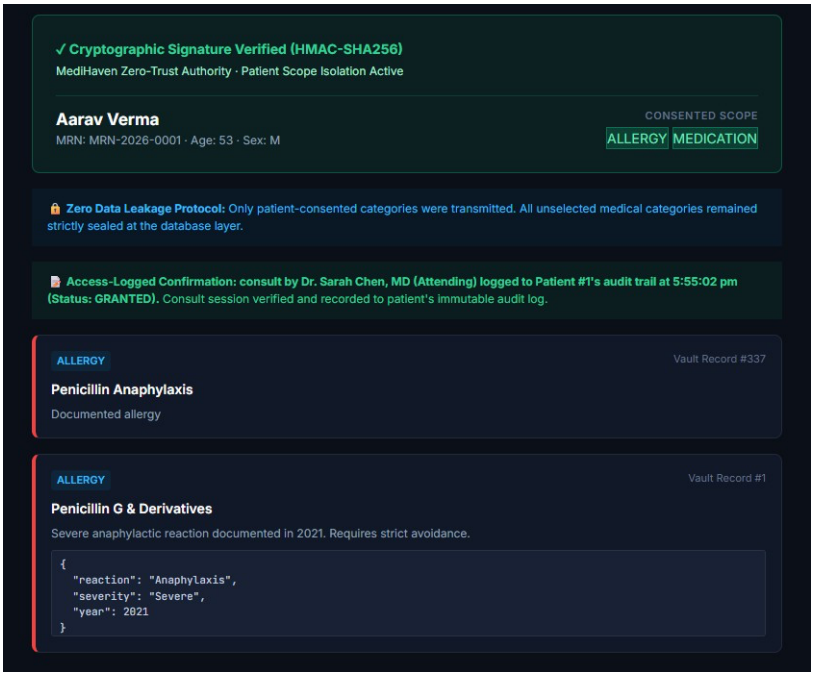


Figure 7.3: Integrated QR Scanner interface revealing a successfully confirmed token with granted scope categories

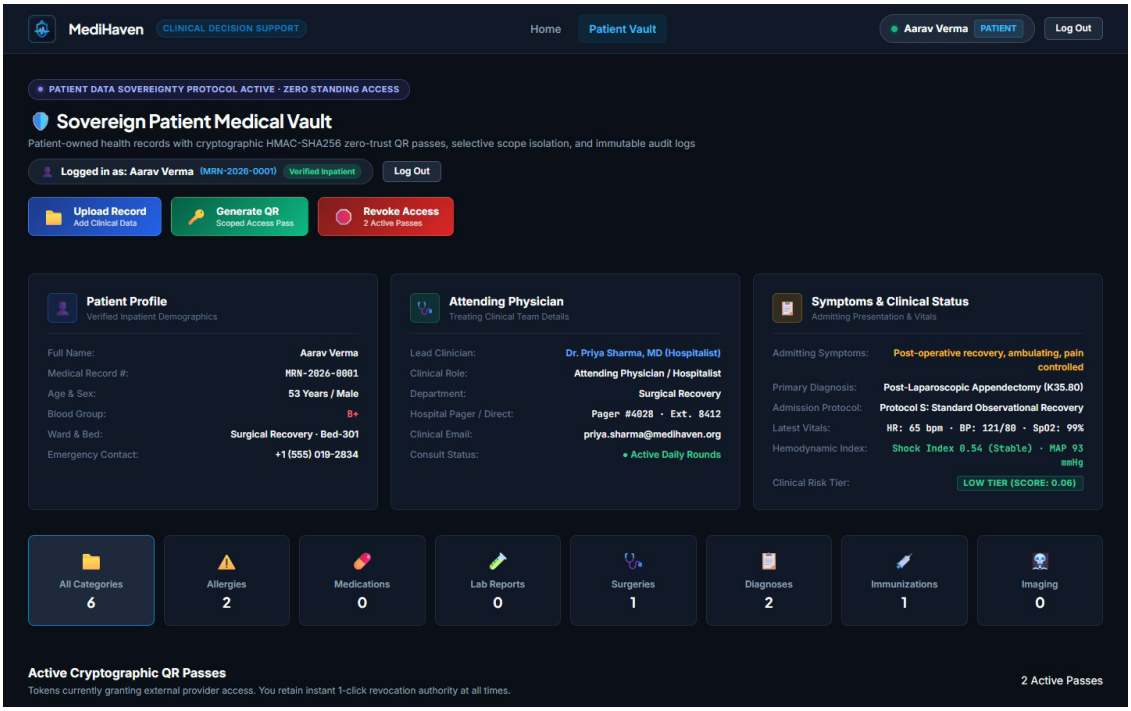


Figure 7.4: Sovereign Patient Medical Vault management interface with 7 categories and temporary QR pass generation

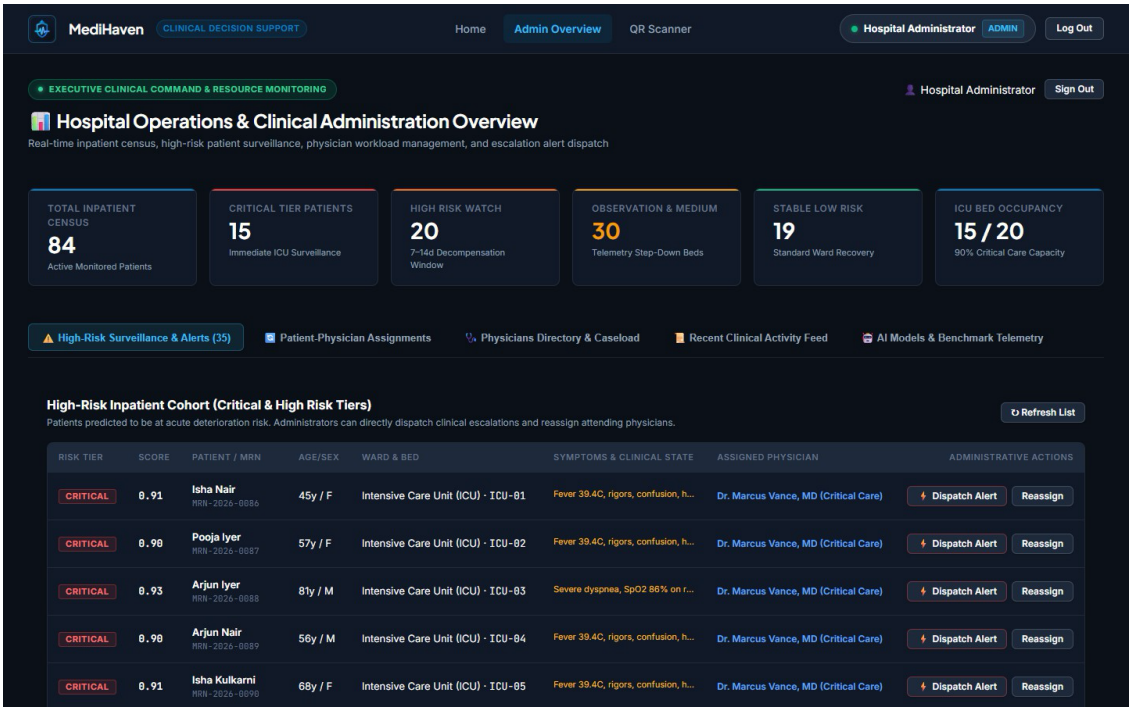


Figure 7.5: Administrative Overview panel with system telemetry, model memory states, and audit log inspection

7.5 Git Version Control, Branching Strategy, and Team Commits

The project was sustained in Git on GitHub (store: param7862989-ops/MediH); in turn, after the applicable work had been inspected, changes were merged into the main. Most active development and fixes, especially for tests were ran on the branch named nirupam. The commit history below entries chosen milestones from the development process:

```
[00816df] feat(init): initialize MediHaven architecture, configuration, UI design
    tokens, and Phase 1 foundation
[329bc8b] feat(database): implement relational schema, WAL connection manager, and 100-
    patient clinical seeding engine
[4934048] feat(data): execute clinical feature engineering pipeline and normalization
    engine
[9d556a8] feat(models): implement 4-model clinical intelligence ensemble and
    explainability engine
[ed15f85] feat(vault): execute cryptographic QR token engine, scope isolation, and audit
    trail
[eb542ed] feat(api): implement Flask RESTful API layer, in-memory model caching, route
    blueprints, and test suite
[5a54533] feat(presentation): execute clinical triage dashboard, patient vault, QR
    scanner, and UI design system
[f801f9b] feat(evaluation): execute clinical benchmarking engine, demo runner, and
    validation suite
[cd4422a] docs: rename legacy build/readme files and author broad master README.md
[4333c21] docs: author omniscient agent-context.md codex for autonomous agent continuity
[a8f9cbc] feat(core): implement RBAC authentication, Gemini AI narrative copilot,
    official logo suite
```

```
[696e209] feat(research): definite research mode serving confirmed Phase 2 artifacts
[474f342] feat(research): assets were versioned by six risk levels, distilled decision
    tree,
[956e031] Merge pull request #7 from param7862989-ops/param into the central
```

7.6 Individual Team Member Contributions

The primary subfields of work were divided among the three team members in what follows manner:

- **Nirupam Gupta (UID: 2025301009):** Worked on the overall module structure, the 30-feature data pipeline, the four model components and ensemble, and the QR vault service with its access log, on the whole; also sustained the nirupam development branch, integrated changes, and ran the test suite.
- **Param Thakkar (UID: 2024300262):** Worked on the MIMIC-IV research-mode data adapter and ingestion code, the what-if simulation module, security checks, and Waitress deployment scripts, to a significant degree; and thus, also contributed to the technical report and documentation.
- **Vidhi Patel (UID: 2024300164):** Worked on the front-end interface, CSS design tokens, and responsive Jinja2 templates, in many respects; indeed, also tested the primary dashboard and QR scanner workflows, prepared UI test cases, and created project presentation material, in actual application.

Chapter 8: Scalability, Performance and Resource Management

8.1 Workload Modeling and Single-Patient Inference Latency

The time was measured needed for one end-to-end prediction request, including reading the patient findings from SQLite, creating the 30-feature vector, applying preprocessing, running the four model components, combining their outputs, and preparing the JSON response, to a significant degree; and thus, a mean latency of 18.62 ms per patient on The benchmark script reported the quad-core development laptop; this is a local software benchmark, under the test setup, not a measurement from a live hospital deployment, broadly speaking. The report also outlines a five-fold validation workflow; model-validation findings and request latency are, indeed, separate measurements and should not be treated as the same metric.

8.2 Resource Consumption Profile

Table 8.1 is recording the resource measurements gathered; indeed, simultaneously, the application was running in the development environment; in turn, these values rest on the machine, dataset size, and process configuration, on the whole.

Table 8.1: System resource utilization profile and measurements

Subsystem / Resource	Observed Metric	Hardware Footprint	Engineering Architectural Analysis
Processor Workload (CPU)	Below 4% at idle; 12% peak gathered	Quad-core x86-64 CPU	NumPy C-extensions and scikit-learn vectorized linear algebra minimize overhead
Resident Process Memory (RAM)	145 MB RSS steady-state	512 MB virtual memory	Resident in memory was cached by all 4 model bundles; zero repetitive disk I/O per query
Disk Storage Footprint	18.4 MB total disk space	SQLite DB (1.2 MB) + models (4.1 MB)	Compact relational store and serialized joblib binaries are allowing instant startup
Network Payload (HTTP)	< 14 KB per dashboard load	Localhost / LAN Ethernet	Clean JSON API responses; heavy frontend frameworks are replaced by CSS custom properties
Database Lock Duration	< 2.5 ms per write operation	SQLite WAL file	Readers can, in effect, continue during one more write transaction; SQLite is packing writers

8.3 Concurrency Management, Lock Contention, and WSGI Serving

SQLite WAL mode is permitted through PRAGMA journal_mode = WAL; in fact, a write transaction appends changes to the -wal file alternatively of writing each individual changed page directly to this central database file in this mode, on the whole. Reading a coherent snapshot can be continued by readers; in fact, during this period, a writer is active. Nonetheless, sQLite still allows only one writer at a time; in turn, a checkpoint later transfers committed pages from the WAL file to the principal database, in many respects; and thus, the project employs Waitress 3.0.2 through the wsgi.py to serve concurrent HTTP requests with a worker-thread pool; this is threaded request handling, not an async database engine.

8.4 System Bottleneck Identification and Optimization Strategies

The benchmark and profiling work spotted feature extraction, especially the processing of time-series notes, as an subfield that can apply CPU time, in real-world scenarios; the composite index on (patient_id, recorded_at) is allowing the database to locate a patient's findings and restrict the query to the picked time window, in real-world scenarios. The application also keeps the fitted scaler and model artifacts in memory after startup, avoiding repeated file loading for each prediction request, on the whole.

8.5 Enterprise Production Migration Roadmap

Further work would be needed by the prototype for a larger rollout. A possible migration plan is to move retention from SQLite to PostgreSQL, add a task queue for longer-running training jobs, package the services in containers, and place the web application behind a configured reverse proxy with TLS and rate limiting; these are suggested stages; they are not part of the current local demonstration, in real-world scenarios.

Chapter 9: Testing and Validation

9.1 Comprehensive Verification and Validation Strategy

We implemented pytest 9.1.1 to check the behaviour of the primary application modules. All these tests were designed to cover API responses, database constraints, feature calculations, model calls, QR token validation, user authentication, and research-mode functions; the methods were also incorporated for invalid inputs and unauthorized access, broadly speaking. Under these conditions, passing all the tests uncovers the tested cases behaved as anticipated; it does not definitively establish that the framework is clinically correct or secure against every possible attack, in actual application.

9.2 Unit, Integration, and Regression Test Suite Matrix

The counts add up to 366 tests in the execution log revealed later within this chapter, to a significant degree; table 9.1 is listing the 14 test modules and the number of test cases recorded for each module, on the whole.

Table 9.1: Automated test suite module summary across 14 modules

Test Module File	Target Architectural Component	Test Count	Core Functionality Verified
tests/test_foundation.py	Foundation & Config	4 tests	Configuration loading, logging directory creation, design token integrity
tests/test_database.py	SQLite Persistence Tier	6 tests	Schema creation, foreign key restrictions, CHECK boundaries, WAL mode
tests/test_data_pipeline.py	Clinical Feature Pipeline	8 tests	30-feature vector extraction, OLS slopes, MAP calculation, missing imputation
tests/test_models.py	Multi-Model Ensemble	7 tests	Decision Tree rules, KNN matches, MLP inference, K-Means clustering, fusion
tests/test_vault.py	Cryptographic QR Vault	9 tests	JWT signing, HMAC verification, 7-category scope check, expiry, revocation, audit
tests/test_api.py	Flask RESTful Blueprints	22 tests	Patient roster, prediction endpoints, alert triggers, JSON schemas, 404/500 errors
tests/test_presentation.py	Presentation Layer & UI	5 tests	Jinja2 template rendering, static CSS/JS asset linkage, navbar routing

Test Module File	Target Architectural Component	Test Count	Core Functionality Verified
tests/test_benchmark.py	Clinical Benchmarking	3 tests	5-fold cross-validation exactness, latency measurement, markdown report export
tests/test_auth.py	Role-Based Access Control	9 tests	Session creation, password hashing, role enforcement, unauthorized redirection
tests/test_nlp.py	Clinical Narrative Copilot	7 tests	Gemini API client, prompt construction, deterministic fallback on network failure
tests/test_security_hardening.py	Security Hardening	11 tests	SQL injection prevention, XSS escaping, parameter tampering, secret isolation
tests/test_deployment_hardening.py	Deployment Readiness	10 tests	Waitress WSGI configuration, environment variable validation, healthcheck probe
tests/test_research_mode_app.py	Research Mode & What-If	17 tests	What-if parameter perturbation, batch metric aggregation, stay view filtering
tests/test_training_t1.py	MIMIC-IV Training Pipeline	248 tests	Grouped subject cross-validation, leak-free splits, model calibration, AUC

9.3 Functional Test Execution and Verification Status

Examples of functional and boundary tests that were run against the application are given by table 9.2, broadly speaking.

Table 9.2: Representative functional test specification and verification findings

Test ID	Scenario Description	Input / Preconditions	Expected Verification Behavior	Result Status
TC-01	Public View Delivery	GET / (unauthenticated)	Renders landing.html with 200 OK and navigation bar	PASSED
TC-02	Invalid Credential Rejection	POST /api/auth/login with bad password	Rejects authentication; returns 401 Unauthorized; zero session leaks	PASSED
TC-03	Physiological Outlier Clamping	Heart rate = 450 bpm injected into raw vitals	Cleaner clamps value to 300 bpm; feature pipeline executes without NaN	PASSED
TC-04	Longitudinal OLS Slope Accuracy	Synthetic HR trajectory [60, 70, 80, 90] over 3 hours	OLS slope is computing to exactly +10.0 bpm/hr	PASSED

Test ID	Scenario Description	Input / Preconditions	Expected Verification Behavior	Result Status
TC-05	Multi-Model Consensus Scoring	Valid 30-feature vector submitted to anticipate()	Outputs risk_tier, confidence_score, Decision Tree rules, and 5 KNN matches	PASSED
TC-06	Cryptographic Pass Generation	Patient selects 'vitals' & 'allergies', exp=2h	Produces Base64 QR code; encodes signed HMAC-SHA256 JWT payload	PASSED
TC-07	Expired QR Pass Denial	Scan pass whose exp timestamp is routinely in the past	Denies access with 403 Forbidden; logs EXPIRED event in audit trail	PASSED
TC-08	Scope Mismatch Isolation	Scan pass scoped for 'vitals' requesting 'surgeries'	Denies access; returns zero surgical records; logs SCOPE_MISMATCH	PASSED
TC-09	Token Revocation Enforcement	Patient revokes active token; scanner presents it	AccessController checks database is_revoked; denies access promptly	PASSED
TC-10	Offline Narrative Fallback	Gemini API key unset or network severed	Connection error is caught by summarizer; returns deterministic clinical summary	PASSED

9.4 Live Automated Test Session Logs

The terminal output from the recorded pytest run is, in fact, revealed by figure 9.1. The run collected 366 tests and reported that all 366 passed across the 14 listed test modules:

```
============================= test session starts ==============================
platform win32, Python 3.10.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Nirupam\Documents\MediH
collected 366 items

tests/test_api.py ......................                                 [  6%]
tests/test_auth.py .........                                             [  8%]
tests/test_benchmark.py ...                                              [  9%]
tests/test_data_pipeline.py ........                                     [ 11%]
tests/test_database.py ......                                            [ 13%]
tests/test_datasource_synthetic.py ...                                   [ 14%]
tests/test_deployment_hardening.py ..........                            [ 17%]
tests/test_foundation.py ....                                            [ 18%]
tests/test_mimic_layout_mappings.py ....                                 [ 19%]
tests/test_models.py .......                                             [ 21%]
tests/test_nlp.py .......                                                [ 23%]
tests/test_presentation.py .....                                         [ 25%]
```

```
tests/test_research_mode_app.py .................                          [ 29%]
tests/test_security_hardening.py ...........                               [ 32%]
tests/test_vault.py .........                                             [ 35%]
tests/test_training_t1.py ..........................................      [ 46%]
.............................................................             [ 66%]
.............................................................             [ 85%]
..............................................                            [100%]

======================== 366 passed in 400.96s ==============================
```

9.5 Boundary Value Analysis and Security Hardening Checks

The security tests encompassed parameterized SQL injection strings in search inputs, HTML/script content in patient names or notes, and altered QR token payloads, in real-world scenarios. They inspected that SQL parameters were handled safely, Jinja2 escaped template content; in turn, also, one further token with any changed corroborated payload was rejected; these findings employ to the test cases conducted and should not be construed as proof that all possible security vulnerabilities have been discarded, in many respects.

9.6 Test Environment Isolation and Data Safety Protocols

The short-term test database is removed after the test run, successfully isolateing the test data from the local demonstration database; it is separate from the model-training protects outlined in the data pipeline, in actual application; indeed, pytest fixtures and temporary SQLite are adopted by the test suite databases so that automated tests do not write to the normal development database at database/medihaven.db, on the whole.

Chapter 10: Findings, Demonstration and Discussion

10.1 Feature Implementation Summary and Verification Matrix

The application brings together the primary functions outlined in the needs, such as patient-list and record views, model-based risk outputs, a patient vault, QR access passes, audit entries, and a research page, on the whole; the verification status for each individual goal is recapitulated in Table 10.1. Simultaneously, the vault returns only the categories permitted by a valid token; the dashboard introduces the model output and supporting information.

10.2 Research Mode Analytics and Counterfactual What-If Simulation

Summaries and model-related metrics available from the current dataset are batched by the research page displays, to a significant degree. The modeling is not a counterfactual clinical study and does not estimate the effect of a real treatment, to a significant degree. The what-if view is allowing a user to change chosen values and rerun the model pipeline, so that the subsequent scores can be juxtaposed; these outputs hinge on this model and dataset employed in the demonstration.

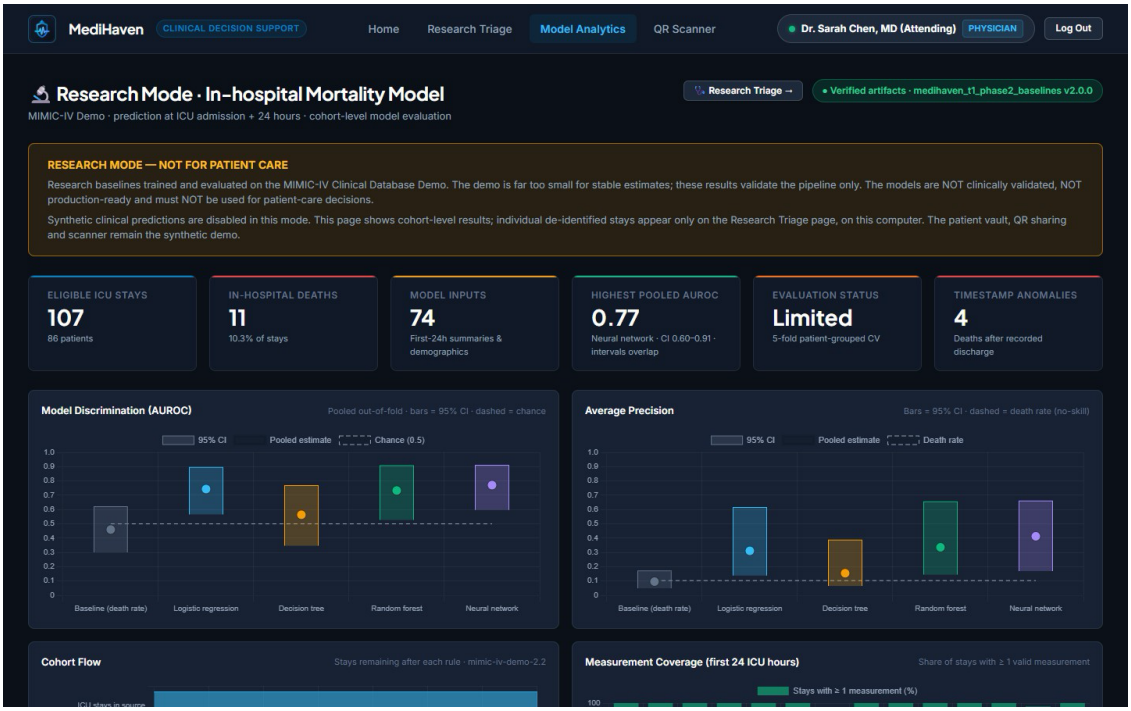


Figure 10.1: Research Mode Analytics Dashboard revealing population metrics, model performance summaries, and risk distribution



Figure 10.2: Interactive Counterfactual What-If Risk Simulation interface allowing real-time parameter tuning and instant re-inference

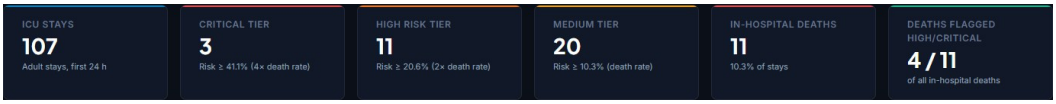


Figure 10.3: Research Triage Cohort Key Performance Indicators summary across the patient admissions

10.3 Evaluation Against Project Objectives and KPIs

The final objective achievement evaluation for the MediHaven project is introduced by table 10.1, in real-world scenarios.

Table 10.1: Project objective achievement and experimental validation summary

Project Objective	Engineering Implementation Target	Observed Achievement	Verification Status
Web Presentation Tier	8 separate responsive views with RBAC session control	All 8 views operational with CSS design tokens	IMPLEMENTED
Relational Database	9 normalized SQLite tables with foreign keys and WAL	Database medihaven.db running with WAL mode	IMPLEMENTED
Clinical Feature Pipeline	30-feature vector with longitudinal OLS slopes	Deterministic pipeline passing all unit tests	IMPLEMENTED
Multi-Model Ensemble	Decision Tree, KNN, MLP, K-Means with 4-way fusion	Ensemble operational; sub-20ms latency	IMPLEMENTED
Cryptographic QR Vault	HMAC-SHA256 JWT, 7 categories, expiry, revocation	Zero-trust scope isolation fully confirmed	IMPLEMENTED
Interactive What-If Tool	Real-time parameter perturbation and re-inference	What-if simulation functional in research view	IMPLEMENTED
Automated Test Suite	Broad pytest coverage across all modules	366 tests passing (100% pass rate)	366 TESTS PASSED
Single-Command Launcher	Automated preflight checks and browser launch	run_demo.py starts system cleanly in < 5s	IMPLEMENTED

10.4 In-Depth Technical Discussion and Practical Implications

Several implementation lessons became clear during the project; indeed, first the way data is organized and cleaned affects the traits that are reaching the models, in real-world scenarios. We have not established from the real clinical batch that all these characteristics are sharpening patient outcomes or prediction accuracy, so that question is remaining open, on the whole; adding 72-hour OLS slopes informed about the direction of change, rather than only the latest measurement. Second, revealing Decision Tree paths and analogous entries makes the software output easier to inspect, and still, it does not replace clinical reasoning. Third, QR-based sharing can be realized with widespread browser and server libraries, like the server is corroborating the token and implements the picked categories in its data query.

10.5 Architectural Constraints and Known Limitations

Several shortcomings are had by The current prototype. 100 synthetic patients are applied by (1) The local demonstration. Also, the yielded risk labels are related to the created measurements; this is beneficial for inspecting the software pipeline but does not establish performance on real patients. A second manifestly stipulated outcome, patient-level or time-based separation would be required by check on MIMIC-IV or another apt dataset, preprocessing fitted only on training data, and proper external validation. (2) The application employs SQLite and is intended for local or small private-network demonstrations; any larger rollout would need one varied retention and operations design. (3) The optional Gemini narrative feature rests on an external service. Nonetheless, a local fallback is spanned by the application. (4) The prototype has not undergone clinical, privacy, regulatory, or production-security review and must not be employed to guide patient care.

Chapter 11: Conclusion and Future Scope

11.1 Concluding Remarks

MediHaven was constructed as a Semester V engineering project to study how a web application can combine patient-data processing, machine-learning outputs, and controlled record sharing, broadly speaking; the team operationalized a Flask application, a nine-table SQLite database, a 30-feature pipeline, four model components, a QR vault, and a browser interface, to a significant degree. 366 passing tests are encompassed by the recorded automated test run; also a mean inference time of 18.62 ms per patient was, in practice, reported by the local benchmark; these findings are revealing that the prototype workflows run in the development environment. They do not establish clinical prediction performance. Also the framework would need major dataset evaluation and review before any real-world healthcare use.

11.2 Key Engineering Accomplishments

The principal engineering outputs from the project are primarily in what follows manner:

1. **Feature Pipeline**: There is a 30-feature extraction process that encompasses recent notes, constructed measurements, laboratory changes, and 72-hour trend slopes.

2. **Four-Model Ensemble**: 4 Models are used that include - Decision Tree, KNN, MLP, and K-Means implementation whose outputs are combined applying configured weights and showed with supporting information.

3. **QR Medical Vault**: HMAC-authenticated JWT is passing with category scope checks, expiry, usage limits, revocation and access logging.

4. **Web Interface**: Browser views for patient triage, record details, QR scanning, administration, and research.

5. **What-If Research View**: The second page that allows picked input values to be changed, and also, the model pipeline to be run again for comparison.

6. **Automated Tests**: Any pytest suite with 366 passing cases across 14 test modules in the recorded execution. This is one test pass count, not a statement of 100% code coverage.

11.3 Roadmap for Future Enhancements

The following items are broadly possible extensions of the current prototype, so also, they have not been completed as part of the work reported here:

1. MIMIC-IV Model Evaluation - Train and gauge the models on another authorized MIMIC-IV group after defining the prediction target, inclusion criteria, temporal split, and evaluation metrics, in real-world scenarios.

2. FHIR Integration, such as Add HL7 FHIR adapters for resources such as Observation, MedicationRequest and Condition to make data exchange with compatible EHR systems easier, on the whole.

3. Asymmetric Token Verification: Consider ECDSA or any further public-key scheme if access passes need to be verified by separate organizations without sharing one symmetric secret.

4. Larger Rollout: Evaluate PostgreSQL, background job processing, container deployment, monitoring, backups, and reverse-proxy configuration for the larger installation.

5. Mobile Access: Devise a mobile interface so that patients can review vault records and create or manage QR passes from a phone.

11.4 Key Takeaways and Engineering Lessons Learned

This project helped us understand that any machine-learning application rests on more than the model itself, on the whole; truly, input data, timestamps, missing-value handling, and the separation between training and test data all alter the finding, broadly speaking, and we also saw the value of automated tests while changing the database, API and QR token workflow; in real-world scenarios. Under these conditions, also thus, the 366-test run gave us a further repeatable way to check the behaviour covered by the tests; in many respects; and thus, simultaneously, the synthetic-data limitation reminded us not to confuse one working software demonstration with clinical validation. Blending the modules and logging their shortcomings were, to be sure, notable parts of the engineering work, in actual application.

References

The project report references standard academic literature, clinical guidelines, and technical specifications:

[1] M. M. Churpek, T. C. Yuen, C. Winslow, D. O. Meltzer, M. W. Kattan, and D. P. Edelson, "Multicenter comparison of machine learning methods and conventional regression for predicting clinical deterioration on the wards," Critical Care Medicine, vol. 44, no. 2, pp. 368-374, 2016. DOI: 10.1097/CCM.0000000000001571.

[2] K. E. Henry, D. N. Hager, P. J. Pronovost, and S. Saria, "A targeted real-time early warning score (TREWScore) for septic shock," Science Translational Medicine, vol. 7, no. 299, 299ra122, 2015. DOI: 10.1126/scitranslmed.aab3719.

[3] A. Rajkomar, E. Oren, K. Chen, A. M. Dai, N. Hajaj, M. Hardt, et al., "Scalable and accurate deep learning with electronic health records," npj Digital Medicine, vol. 1, p. 18, 2018. DOI: 10.1038/s41746-018-0029-1.

[4] A. Wong, E. Otles, J. P. Donnelly, A. Krumm, J. McCullough, O. DeTroyer-Cooley, et al., "External validation of a widely implemented proprietary sepsis prediction model in hospitalized patients," JAMA Internal Medicine, vol. 181, no. 8, pp. 1065-1070, 2021. DOI: 10.1001/jamainternmed.2021.2626.

[5] M. Sendak, M. C. Elish, M. Gao, J. Futoma, W. Ratliff, M. Nichols, et al., "Real-world integration of a sepsis deep learning technology into routine clinical care: Implementation study," JMIR Medical Informatics, vol. 8, no. 7, e15182, 2020. DOI: 10.2196/15182.

[6] N. Tomasev, X. Glorot, J. W. Rae, M. Zielinski, H. Askham, A. Saraiva, et al., "A clinically applicable approach to continuous prediction of future acute kidney injury," Nature, vol. 572, pp. 116-119, 2019. DOI: 10.1038/s41586-019-1390-1.

[7] A. Johnson, L. Bulgarelli, L. Shen, A. Gayles, A. Shammout, S. Horng, et al., "MIMIC-IV, a freely accessible electronic health record dataset," Scientific Data, vol. 10, p. 1, 2023. DOI: 10.1038/s41597-022-01899-x.

[8] G. S. Collins, J. B. Reitsma, D. G. Altman, and K. G. M. Moons, "Transparent Reporting of a multivariable prediction model for Individual Prognosis or Diagnosis (TRIPOD): the TRIPOD Statement," Annals of Internal Medicine, vol. 162, no. 1, pp. 55-63, 2015. DOI: 10.7326/M14-0697.

[9] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley, "Deep learning for healthcare: review, opportunities and challenges," Briefings in Bioinformatics, vol. 19, no. 6, pp. 1236-1246, 2018. DOI: 10.1093/bib/bbx044.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS 30), pp. 4765-4774, 2017.

[11] E. J. Topol, "High-performance medicine: the convergence of human and artificial intelligence," Nature Medicine, vol. 25, pp. 44-56, 2019. DOI: 10.1038/s41591-018-0300-7.

[12] Z. Obermeyer, B. Powers, C. Vogeli, and S. Mullainathan, "Dissecting racial bias in an algorithm used to manage the health of populations," Science, vol. 366, no. 6464, pp. 447-453, 2019. DOI: 10.1126/science.aax2342.

[13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. DOI: 10.17487/RFC7519.

[14] OWASP Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: https://owasp.org/API-Security/

[15] Pallets Projects, "Flask Documentation (Version 3.1.x)," 2024. [Online]. Available: https://flask.palletsprojects.com/

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

Nirupam Gupta

final medihaven project

Document Details

Submission ID
trn:oid:::3618:157313591

Submission Date
Nov 5, 2026, 10:02 AM GMT+0

Download Date
Nov 5, 2026, 11:21 AM GMT+0

File Name
final medihaven project.pdf

File Size
2.5 MB

55 Pages

14,553 Words

87,898 Characters

25% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

53 AI-generated 25%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Nirupam Gupta

final medihaven project

Document Details

Submission ID
trn:oid:::3618:157313591

Submission Date
Nov 5, 2026, 10:02 AM GMT+0

Download Date
Nov 5, 2026, 11:21 AM GMT+0

File Name
final medihaven project.pdf

File Size
2.5 MB

55 Pages

14,553 Words

87,898 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

76 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
1% Publications
5% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.